



Language and AI

Why is language difficult?

Ambiguity

Common Sense & World Knowledge

How We Learn Language (and How Machines Do)

A Brief History of NLP and "The Bitter Lesson"

Text mining preliminaries

From Text to Vectors

TF-IDF

Normalization and Cosine Similarity

Cosine Similarity

Exercises

Collecting Data

Text Pre-Processing: The "Why" and "How" (noisy text)

Regular Expressions (Regex)

System Evaluation

Normalization

Classification

N-grams Language Models

Maximum Likelihood Estimation (MLE)

Evaluating LMs: Perplexity

Handling Zeros & Smoothing

Naive Bayes Classification

Decision Trees (ID3)

Regression Models (Linear & Logistic)

Support Vector Machines (SVM)

k-Nearest Neighbors (k-NN)

Generalization & Evaluation

Representation

Meaning as Counts: Distributional Semantics

Distributional Similarity (and PMI)

Dimensionality Reduction with LSA / SVD

Predicting Meaning: Neural Word Embeddings

Word2Vec: CBOW and Skip-Gram

Skip-Gram with Negative Sampling (SGNS)

Properties of Embeddings

Evaluation

Neural Models of Language

Practical Applications: Evaluation, Debugging, and Robustness (Expanded)

[Information Extraction](#)

[Where Can Information Live?](#)

[Motivating Examples](#)

[Sequence Classification: The General Setup](#)

[Sequence Models: Markov Models & Hidden Markov Models](#)

[Hidden Markov Models \(HMMs\)](#)

[Decoding](#)

[Decoding: The Viterbi Algorithm](#)

[Conditional Random Fields \(CRFs\)](#)

[Feature Templates](#)

[Information Extraction: Applications](#)

[Deep Learning](#)

[Recurrent Neural Networks](#)

[RNNs as Language Models](#)

[Generalizing RNN architecture to other tasks](#)

[LSTM](#)

[Encoder-Decoder Models](#)

[Machine translation](#)

[The Solution: Attention](#)

[Transformers](#)

[Encoder-Decoder in Transformers](#)

[GPT](#)

[BERT](#)

[Large Language Models, ChatGPT, and Prompt Engineering](#)

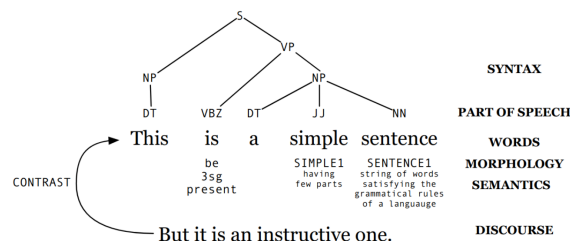
[InstructGPT](#)

[Prompt engineering](#)

Why is language difficult?

This lecture explores why language is so difficult for computational processing. While on the surface, language might seem like a simple combination of nouns (objects), verbs (actions), and adjectives (properties), this is a very reductionist view. In reality, language is an extremely rich information medium with many categories and edge cases.

To understand its complexity, we can break a simple sentence down into multiple layers of information:



- Surface Form (Words):** This is the observable layer — the words themselves, separated by whitespace. However, the words alone don't explain the role they play in communicating information.
- Morphology (Word Forms):** This layer analyzes how words are related. For example, the word "is" is an inflection of the verb "to be," indicating it's in the present tense.
- Part-of-Speech (POS) Tags (Word Categories):** This adds another layer of information, categorizing words into classes like nouns (NN), verbs (VBZ), adjectives (JJ), and determiners (DT).
- Syntax (Phrase Structure):** This layer defines the grammatical structure and how words connect to each other, often represented as a tree. For instance, "a simple sentence" forms a "noun phrase" (NP).
- Semantics (Word Meaning):** This is the interpretation or meaning of the words. This is where ambiguity becomes a major problem.
 - "Simple" could mean "having few parts" or it could be a pejorative term for a person.
 - "Sentence" could mean a string of words or a prison sentence.
- Discourse (Contextual Meaning):** This final layer looks at the broader context. A sentence like, "This is a simple sentence, **but** it is an instructive one," introduces a contrast relation that adds meaning on top of the sentence itself.

Two of the biggest reasons language is so difficult are ambiguity and the reliance on world knowledge.

Ambiguity

Sentences can often be interpreted in multiple ways, and the correct syntactic structure is unclear.

- **"They saw a kid with a telescope."**
 - *Interpretation 1:* They used a telescope to see the kid.
 - *Interpretation 2:* They saw a kid who was holding a telescope.
- **"Flying planes can be dangerous."**
 - *Interpretation 1 (Adjective):* Planes that are currently flying are dangerous objects.
 - *Interpretation 2 (Verb):* The act of flying a plane is a dangerous activity.
- **"Time flies like an arrow."**
 - *Interpretation 1 (Idiom):* Time passes very quickly.
 - *Interpretation 2 (Noun):* A specific type of insect called a "time fly" has a fondness for arrows.
 - *Interpretation 3 (Verb):* A command to use a stopwatch to "time" the flies in the same way you would time an arrow.

Because of this, language is always **contextualized**, and a key goal of NLP is to find ways to represent this context and structure.

Common Sense & World Knowledge

Humans constantly use vast amounts of common sense and world knowledge to understand language, which machines lack.

This is very clear in **sentiment analysis**. A system can be easily fooled if it only looks at surface-level words:

- **Negation:** In "This movie is definitely **not bad**," the model must understand that "not bad" is actually positive.
- **Sarcasm:** A sentence like "This is the **best movie ever**, lol" appears positive ("best movie ever"), but the sarcasm indicated by "lol" or an emoji flips the meaning entirely.
- **Dependencies:** In "**Whoever thinks this film is incredibly bad** is an idiot," a simple model would see "incredibly bad" and classify the sentiment as negative. It needs to understand the full syntactic structure to see that the speaker is *contrasting* this opinion.
- **World Knowledge:** To understand "This movie is only slightly better than **Jaws**," the system needs the external world knowledge that *Jaws* is a critically acclaimed and highly-rated film.

How We Learn Language (and How Machines Do)

If language is so hard, how do humans learn it?

A "naive engineer" approach to machine learning mimics this process by simply learning patterns from text. You can "chuck a whole bunch of text at a model," give it a goal (like recognizing words), and provide feedback.

However, **psycholinguistics** research into how children *actually* learn shows a more complex process:

1. **Segmentation:** First, babies must learn to segment the continuous string of speech they hear into individual words. They use cues like rhythm and stress (e.g., most English words have initial stress). This is difficult, as audio waveforms don't have clean breaks between words.
2. **Mapping:** Once they can segment words, they start mapping these tokens to objects in the world.
3. **Vocabulary Spurt:** Around age two, a "vocabulary spurt" occurs. They learn new words rapidly by placing them in the context of words they already know (a process called "fast mapping").

Crucially, humans use **grounded learning**: we learn language by connecting it to visuals, objects, and speech — not just by processing text in isolation. This has inspired multimodal research in AI that combines images, speech, and text.

A Brief History of NLP and "The Bitter Lesson"

You might think that linguistics and psycholinguistics are central to developing NLP systems. However, the history of the field shows a different story, summarized by "The Bitter Lesson": smart, human-designed "tricks" and analyses are almost always beaten by simpler systems that just use **more computational power** and **more data**.

- **Up to 1970s:** The field was dominated by Noam Chomsky's theory that language is innate. Researchers built complex grammars and ontologies by hand.

- **1970s-80s (AI Winter):** AI was over-hyped and failed to deliver. Machine translation was more expensive than human translators, leading to massive funding cuts.
- **1980s-2010s:** Researchers turned to **statistical linguistics** and learning from data. This era was defined by cool, but very problem-specific, algorithms.
- **2010s-2020s: Deep Learning** (big neural networks) took over. This was a difficult, time-intensive era where models were trained for one specific task and could not be re-used.
- **2020s-Now: Transformers** still dominate. The key difference is that we can now train huge, general-purpose systems on various tasks. This allows major companies to train a massive model, which "humble researchers" can then download and use.

Even though "The Bitter Lesson" holds, (psycho)linguistics is still vital for addressing the remaining problems in modern NLP:

- **Interpretability:** Modern systems are "black boxes." We need linguistic questions to understand *why* they make certain decisions.
- **Fairness:** Models are trained on human data, which is inherently **biased**. This can cause systems to produce harmful or unethical content.
- **Consistency:** Models are still imperfect and require debugging. To test them, we need expert knowledge to create good test sets. This is why most deployed systems (like chatbots or stock predictors) are put in "heuristic prisons" to limit their risk.

Text mining preliminaries

This part introduces the basics of text mining, moving from the *why* of language difficulty to the *how* of representing it in a computable format.

For any text mining task, whether it's classification or retrieval, we need **good language representations**. The goal is to represent **semantics** (meaning).

From Text to Vectors

Language is stored as strings of characters, which have some inherent structure (e.g., capitalization, punctuation, whitespace). To perform calculations, we must convert this text into numbers — specifically into **fixed-size vectors**.

How do we convert a document like "the cat sat on the mat" into a vector?

Bag-of-Words (BoW)

The most common and simplest method is the **Bag-of-Words (BoW)** representation.

- We establish a **vocabulary** of all the words that exist in our document collection.
- We then represent each document as a binary vector, marking a "1" if a word from the vocabulary is present and a "0" if it is not.

Key Limitation: This representation is **unordered**. It loses all structure and grammar, treating the document as just a "bag" of words.

This creates a **document-term matrix**, which we can use for tasks like document retrieval. A simple way to measure similarity here is the **Jaccard Coefficient**, which measures the overlap between two sets of words. It is calculated as the size of the **intersection** (words in both) divided by the size of the **union** (words in either).

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

Wikipedia articles:

data	language	learning	mining	text	vision	<i>y</i>
1	0	1	0	0	1	CV
1	1	1	0	1	0	NLP
1	0	1	1	1	0	TM

- CV = Computer vision
- NLP = Natural Language Processing
- TM = Text Mining

$$\begin{aligned} d_0 &= \langle 1, 0, 1, 0, 0, 1 \rangle & J(d_0, d_1) &= 2/5 = 0.4 \\ d_1 &= \langle 1, 1, 1, 0, 1, 0 \rangle & J(d_0, d_2) &= 2/5 = 0.4 \\ d_2 &= \langle 1, 0, 1, 1, 1, 0 \rangle & J(d_1, d_2) &= 3/5 = 0.6 \end{aligned}$$

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

words in A **and** B (*intersection*) /
words in A **or** B (*union*)

Term Frequency (TF)

Binary representations are compact, but they don't capture the *weight* or *importance* of a word. A better method is to count the **Term**

Frequency (TF) — the number of times a term appears in a document.

- **Document 1:** "the cat sat on the mat" -> `cat: 1`
- **Document 2:** "my cat sat on my cat" -> `cat: 2`

However, raw counts are problematic. A word with a TF of 100 isn't necessarily 10 times more important than a word with a TF of 10. To fix this, we use **log-weighting** (e.g., $\ln(tf + 1)$). This scales down the frequencies, so the difference between TF 100 and TF 110 is much smaller than the difference between TF 1 and TF 11. We add 1 ("smoothing") to avoid taking the log of 0.

```
d0 = 'natural-language-processing.wiki'
d1 = 'information-retrieval.wiki'
d2 = 'artificial-intelligence.wiki'
d3 = 'machine-learning.wiki'
d4 = 'text-mining.wiki'
d5 = 'computer-vision.wiki'
```

$$t = [\text{learning}]$$

$$X_t = \begin{bmatrix} 27 \\ 2 \\ 46 \\ 134 \\ 6 \\ 10 \end{bmatrix} \quad \ln(X_t) = \begin{bmatrix} 3.33 \\ 1.10 \\ 3.85 \\ 4.91 \\ 1.95 \\ 2.40 \end{bmatrix}$$

- $tf = 10$ less important than $tf = 100$, but also $\cdot 10$?
- Information Theory to the rescue!
- Natural log: $\ln(tf(t, d) + 1)$
- Note +1 smoothing to avoid $\ln(0) = -\infty$

Inverse Document Frequency (IDF)

We still have two problems:

1. Longer documents will have higher TF counts, making them seem different from shorter ones, even if they're on the same topic.
2. Common words (like "the" or "and") will have high frequencies but are not informative. **Rare terms should be more important.** If two documents are the only ones in a collection to contain the word "normalization," they are very likely to be similar.

We solve the second problem using **Inverse Document Frequency (IDF)**.

- **Document Frequency (DF):** First, we count how many documents in our *entire collection* a term appears in.
- **Inverse Document Frequency (IDF):** We then invert this logic. The *more* documents a term appears in, the *less* important it is.
- **Formula:** $idf_t = \log_b\left(\frac{N}{df_t}\right)$ (where N is the total number of documents and df_t is the document frequency of term t , i.e. how many documents t appears in at least once).

$$idf_t = \log_b \frac{N}{df_t} \quad N = \text{nr. of documents}$$

$$t = [\text{naive}]$$

$$X_t = \begin{bmatrix} 1 \\ 0 \\ 2 \\ 3 \\ 0 \\ 0 \end{bmatrix} \quad df_t = 3 \quad idf_t = \lg \frac{6}{3} = -0.30$$

TF-IDF

We put this all together by multiplying the (log-weighted) Term Frequency by the Inverse Document Frequency.

$$w_{t,d} = \log(tf(t, d) + 1) \cdot \log_b \frac{N}{df_t}$$

Here, **tf** is the term frequency, or 'how many times does the word/term t occur in a document d '.

Then, **df** is the document frequency, or 'how many documents does the word t occur in'.

So, **tf** is inferred from instances, whereas **df** is inferred from the dataset—i.e., the collection of documents.

N is the total amount of documents, b is a base (typically 10), $\log$ is $\ln$, $\log_{10}$ is $\lg$.

This is a very powerful weighting scheme that creates a high-quality vector representation of the document.

Normalization and Cosine Similarity

We still have the first problem: document length. A long document on a topic will have many high TF-IDF values, while a short document on the *exact same topic* will have fewer. If we use standard **Euclidean distance** to compare them, they will seem very far apart in the vector

space: $\sqrt{\sum_{i=1}^n (\vec{p}_i - \vec{q}_i)^2}$

Solution: L2 Normalization

We fix this by "normalizing" the vectors.

1. We calculate the "length" (or norm) of the entire document vector using the L2 norm formula: $\|\vec{p}\|_2 = \sqrt{\sum p_i^2}$
2. We then **divide every single value in the vector by this norm**.

This process essentially "pushes all the vectors in a circle around the origin," making them all have a length of 1. Now, the vector's *length* (driven by document size) is irrelevant; only its *direction* (topic) matters.

Cosine Similarity

Once our vectors are L2-normalized, we can no longer use Euclidean distance. Instead, we use Cosine Similarity.

- This measures the angle between the two vectors. If they point in the same direction, they are very similar (cosine of $0^\circ = 1$). If they are unrelated (orthogonal), they are not similar (cosine of $90^\circ = 0$).
- When vectors are L2-normalized, this calculation is very simple: it's just the **dot product** of the two vectors.
- If the vectors are *not* normalized, you must use the full formula:

$$\frac{\vec{p} \bullet \vec{q}}{\sqrt{\vec{p} \bullet \vec{p}} \cdot \sqrt{\vec{q} \bullet \vec{q}}} \quad (2)$$



Note that $\sqrt{\vec{p} \bullet \vec{p}}$ is the ℓ_2 norm for vector $\vec{p}$. If you ℓ_2 normalize the full space, the denominator drops, and it's simply $\|\vec{p}\|_2 \bullet \|\vec{q}\|_2$ to get the cosine similarity.

Here $\bullet$ explicitly denotes the dot product 📌

$$\sum_{i=1}^n \vec{p}_i \cdot \vec{q}_i \quad (3)$$

Exercises

🚩 Task 1

Given the following set of vectors:

$$\begin{array}{c|cc} \vec{x}_1 & 6.6 & 6.2 \\ \vec{x}_2 & 9.7 & 9.9 \\ \vec{x}_3 & 1.3 & 2.7 \\ \vec{x}_4 & 1.3 & 1.3 \end{array}$$

- Calculate the Euclidean Distance between vector 1 and vector 4.

▼ Solution (Full)

$$\sqrt{(6.6 - 1.3)^2 + (6.2 - 1.3)^2} = 7.218$$

- Calculate the Cosine Similarity between vector 2 and 3.

▼ Solution (Full)

$$\frac{9.7 \cdot 1.3 + 9.9 \cdot 2.7}{\sqrt{9.7 \cdot 9.7 + 9.9 \cdot 9.9} \cdot \sqrt{1.3 \cdot 1.3 + 2.7 \cdot 2.7}} \approx \frac{39.34}{13.86 \cdot 2.997} \approx 0.947$$

And given:

$$\begin{array}{c|cccccccc} \vec{x}_1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ \vec{x}_2 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \end{array}$$

- Calculate the Jaccard coefficient between vector 1 and 2.

▼ Solution (Full)

$$\frac{4 \text{ (total shared words)}}{7 \text{ (total words in either)}} = 0.571$$

Task 2

- Given the three documents below, calculate the term frequency matrix.

the cat sat on the mat
my cat sat on my cat
my cat sat on the mat on my cat

▼ Solution (Full)

First we determine all the unique words (our vocabulary): the, cat, sat, on, mat, my. These are our features. The documents are our instances. We list the term frequency per document for each feature (term), as such:

#	the	cat	sat	on	mat	my
1	2	1	1	1	1	0
2	0	2	1	1	0	2
3	1	2	1	2	1	2

- What is the df for **cat**, **the** and **my** ?

▼ Solution (Full)

Cat occurs in all 3 documents, the in 2, and my in 2 as well.

- Convert the tf matrix from the previous question to a tf · idf matrix with base 10.

▼ Solution (Full)

Given:

#	the	cat	sat	on	mat	my
1	2	1	1	1	1	0
2	0	2	1	1	0	2
3	1	2	1	2	1	2

For the first row, we have, for example:

$$w_{\text{the},1} = \log(\text{tf}(t, d) + 1) \cdot \log_b \frac{N}{\text{df}_t} = \ln(2 + 1) \cdot \lg \frac{3}{2} = 0.193$$

$$w_{\text{cat},1} = \log(\text{tf}(t, d) + 1) \cdot \log_b \frac{N}{\text{df}_t} = \ln(1 + 1) \cdot \lg \frac{3}{3} = 0.000$$

$$w_{\text{mat},1} = \log(\text{tf}(t, d) + 1) \cdot \log_b \frac{N}{\text{df}_t} = \ln(1 + 1) \cdot \lg \frac{3}{2} = 0.122$$



Tip: because df is static across instances for the same term, you can typically quickly find the same values for these low frequency documents.

#	the	cat	sat	on	mat	my
1	0.194	0.000	0.000	0.000	0.122	0.000
2	0.000	0.000	0.000	0.000	0.000	0.194
3	0.122	0.000	0.000	0.000	0.122	0.194

Collecting Data

Text Pre-Processing: The “Why” and “How” (noisy text)

Text pre-processing converts raw language data into a more uniform and standardized format. The main goal is to **reduce noise and variance** in the data, ensuring that models interpret text consistently.

Data Collection: The Messy Reality

Raw Text

The simplest form of data is raw text, but when extracted from structured sources like HTML (e.g., through web scraping), it can be messy and full of irrelevant elements such as tags, styles, and metadata.

Structured Data

Structured or semi-structured sources are much easier to use. They follow standardized formats and are often accessible through APIs.

Examples of Structured Sources

- Hugging Face Datasets: Provide standardized datasets for NLP tasks like classification, sequence modeling, and translation.
- Platform APIs (Twitter, Wikipedia, Reddit): Offer structured objects, usually in JSON format, containing not just text but additional information like author IDs, timestamps, geolocation, and language.
- Reddit via Pushshift Dumps: A common resource for large-scale, structured Reddit data.

The “Garbage In, Garbage Out” Problem

User-generated text is **creative** and **inconsistent**, introducing variation that complicates automatic processing. If data is not properly cleaned or normalized, models will learn from **noisy, unrepresentative input**.

Example: Impact of Typos

A few spelling errors can greatly increase the size of a vocabulary, since the system treats misspelled words as distinct tokens. This reduces the quality of similarity calculations and model performance.

Common sources of variation include:

- Abbreviations and slang
- Inconsistent capitalization
- Character flooding (e.g., “gooooood”)
- Word concatenations (hashtags, e.g., “#MachineLearningIsCool”)
- Emoticons and emojis
- Dialectal variation and typos

Regular Expressions (Regex)

Regular expressions are a concise, standardized **scripting language for automatically finding, matching, and replacing text patterns**. They are essential tools for cleaning and pre-processing text data.

Functionality	Example Syntax	Explanation
Literal Match	"you"	Finds the exact sequence “you”.
Disjunction	[Yy]ou	Matches either “You” or “you”.
Negation & Range	[^a-z]	Matches anything that is <i>not</i> a lowercase letter.
Kleene Star (*)	o[a-z]*	Matches “o” followed by zero or more lowercase letters.
Kleene Plus (+)	u[a-z]+	Matches “u” followed by one or more lowercase letters.
Wildcard (.)	.	Matches almost any single character.

Functionality	Example Syntax	Explanation
Substitution	<code>re.sub('!+', '!', "Amazing!!!!!!!!!!!!")</code>	Replaces one or more exclamation marks with a single one.

Note: Regular expressions use **greedy matching**, meaning they will always try to match the longest possible string.

```
text = "You there, did you what you were looking for?"
regex_find("[Yy]ou", text)
```

```
<re.match object; span=(0, 3), match='You'>
<re.match object; span=(15, 18), match='you'>
<re.match object; span=(24, 27), match='you'>
```

```
re.sub('!+', '!', "Amazing!!!!!!!!!!!!")
```

```
'Amazing !'
```

Example: ELIZA was an early chatbot that simulated a Rogerian psychotherapist. It achieved this by using regex rules to detect key phrases (e.g., “sad”, “always”, “anxious”) and substitute them into pre-written response templates.

Despite appearing intelligent, ELIZA was purely rule-based, relying on pattern matching and substitution rather than real understanding.

Cheat sheet:

RE	Meaning	Example	Match	No Match
<code>.</code>	Any single character	<code>n.p</code>	nop, nlp	noop
<code>[abc]</code>	Any one of these characters	<code>analy[zs]e</code>	analyze, analyse	analyce
<code>[a-z]</code>	Any characters in this range	<code>test[2-4][a-c]</code>	test2a, test3c	test1a, test2z
<code>[^abc]</code>	Not one of these characters	<code>analy[^\s]e</code>	analyze	analyse
<code>[^a-z]</code>	Not a character in this range	<code>demo[1-3][^a-c]</code>	demo1d, demo2f	demo1a
<code> </code>	Or	<code>test demo</code>	demo, test	example
<code>^</code>	(string) Starts with	<code>^be</code>	be that as it may	to be
<code>\$</code>	(string) Ends with	<code>be\$</code>	that's where i'll be	be there
<code>?</code>	Zero or one time(s) (greedy)	<code>tests?[1-3]</code>	test1, tests2	testx1
<code>??</code>	Zero or one time(s) (lazy)	<code>tests??[1-3]??</code>	test	tests, tests2
<code>*</code>	Zero or more time(s) (greedy)	<code>lo*ve it</code>	loooooove it, lve it	luve it
<code>*?</code>	Zero or more time(s) (lazy)	<code>.*?</code>	see comment	
<code>+</code>	One or more time(s) (greedy)	<code>lo+ve it</code>	love it, loove it	lve it
<code>+?</code>	One or more time(s) (lazy)	<code>lo+?ve it</code>	love it	loove it
<code>{n}</code>	n times	<code>ba{2}</code>	baa	baaa
<code>{n,m}</code>	n to m times	<code>ba{1,2}</code>	ba, baa	baaa
<code>{n,}</code>	n times or more	<code>ba{3,}</code>	baaa, baaaa	baa
<code>()</code>	group	<code>(do get) that</code>	do that, get that	make that
<code>(?:)</code>	passive group	<code>(?:ignore) this</code>	(ignore) this	
<code>(?=)</code>	lookahead	<code>we (?!ignore)</code>	we (ignore)	

RE	Meaning	Example	Match	No Match
<code>(?<=)</code>	lookbehind	<code>(?<=ignore) this</code>	(ignore) this	
<code>\</code>	escape character	<code>really\?</code>	really?	reall
<code>\s</code>	white space	<code>a\s space</code>	a space	aspace
<code>\S</code>	not a white space	<code>\S*</code>	token	
<code>\d</code>	digit	<code>[a-z]\d</code>	a9	aa
<code>\D</code>	not a digit	<code>[a-z]\D</code>	aa	a9
<code>\w</code>	letter	<code>\w+</code>	words	543
<code>\W</code>	not a letter (no alphabet)	<code>\W</code>	!	z

`\b` represents a word boundary.

System Evaluation

Evaluation measures how accurate or reliable a system is and identifies types of errors it makes.

NLP Task Formulations

Task	Notation	Example
Classification (Label)	$f(d) \rightarrow y$	Predict a single label for a document (e.g., spam/not spam, sports/politics/technology...).
Sequence Prediction	$f(d_t) \rightarrow y_t$	Predict a label for each word or token in a sequence (e.g., POS tagging, i.e.: input: “The cat sat on the mat.” Output: <i>DET NOUN VERB PREP DET NOUN</i>).
Information Retrieval	$f(d; q) \rightarrow r$	Given a document (d) and a query (q), the model predicts a relevance score (r) — how well the document answers the query. E.g. Search Engines (e.g., Google): Input: Query (q): “climate change effects on agriculture” Document (d): a web article about global warming and crop yields. Output: A relevance score (r), used to rank results.

Error Types and Confusion Matrix

	Predicted Positive ($\hat{y} = 1$)	Predicted Negative ($\hat{y} = 0$)
Actual Positive (y=1)	True Positive (TP)	False Negative (FN)
Actual Negative (y=0)	False Positive (FP)	True Negative (TN)

Error Definitions

- **False Positive (FP):** The model predicts a positive case incorrectly.
- **False Negative (FN):** The model misses a positive case it should have identified.

Classification Metrics

Metric	Formula	Focus / Use Case
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall correctness; unreliable with imbalanced data.
Precision	$\frac{TP}{TP+FP}$	Penalizes false positives; useful when false alarms are costly (e.g., fraud detection).
Recall	$\frac{TP}{TP+FN}$	Penalizes false negatives; important when missing positives is harmful (e.g., disease detection).
F1-Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Harmonic mean of precision and recall. It balances them.
F_β-score	$\frac{(1+\beta^2) \cdot TP}{(1+\beta^2) \cdot TP + \beta^2 \cdot FN + FP}$	It is a generalized form of the F1-score that allows you to control the balance between

Metric	Formula	Focus / Use Case
		Precision and Recall. When $\beta = 1$, you give equal weight to Precision and Recall → this is the F1-score. When $\beta > 1$, you favor Recall more (you care more about catching all positives). When $\beta < 1$, you favor Precision more (you care more about avoiding false positives).

Why is accuracy not enough? Suppose that we have very imbalanced data, like e-mails where 98% of them are spam; if we take a dummy model that classifies all e-mails as spam, it will get 98% accuracy (very high) even though it is absolutely stupid.

Information Retrieval (IR) Metrics

For Information Retrieval tasks, the evaluation is different from simple classification because you are not predicting a single label, but rather a **ranking** of documents based on their relevance to a query. The system's output is a ranked list, and the metrics must assess the quality of this ranking.

1. Mean Reciprocal Rank (MRR)

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

Where:

- $|Q|$ = number of queries
- $rank_i$ = the rank position of the *first relevant* document for the *i*th query

MRR rewards systems that return relevant results **earlier** in the ranking. A higher MRR means users find relevant answers more quickly.

Example: for three queries, the first relevant documents appear at ranks 1, 3, and 2. $MRR = \frac{1}{3} \left(\frac{1}{1} + \frac{1}{3} + \frac{1}{2} \right) = 0.61$. On average, the first relevant result appears around the **second position**.

2. Precision@K

$$K = \frac{\# \text{ of relevant documents in top } K}{K}$$

Measures **how many of the top K retrieved documents are relevant**.

Order within the top K does not matter — only relevance.

3. Average Precision (AP)

$$AP = \frac{\sum_{k=1}^N (Precision@k \times rel(k))}{\# \text{ of relevant documents}}$$

Where:

- N = total number of retrieved documents
- $rel(k) = 1$ if the document at rank k is relevant, otherwise 0

AP averages the precision at every rank position where a relevant document appears. It combines both **precision** and **ranking quality** into one measure.

Suppose relevant documents appear at ranks 2, 4, and 5:

Rank	Relevant?	Precision@k
2	✓	0.5
4	✓	0.5
5	✓	0.6

Annotation and Inter-Rater Agreement (IRA)

Human annotators provide the “ground truth” labels used for training and evaluation.

Cultural Dependence

Annotations reflect human judgment, which can be culturally dependent and biased (e.g., what counts as “toxic” language varies by demographic context).

Inter-Rater Agreement (IRA)

To ensure label quality, multiple annotators label the same data. Agreement is typically measured using **Cohen's Kappa (κ)**, which adjusts for agreement by chance. A higher κ indicates greater label reliability.

Normalization

Word normalization is the task of putting words or tokens in a standard format.

Normalization reduces unnecessary variation in text to shrink the effective vocabulary and improve model performance.

Vocabulary Variation

In language data, the ratio of unique tokens (**types**) to total tokens (**tokens**) tends to be high.

According to **Heap's Law**: $|V| = k \cdot N^\beta$, where $0 < \beta < 1$

Corpus	Tokens (N)	Types ($ V $)
Shakespeare	884 K	31 K
Brown corpus	1 M	38 K
Switchboard	2.4 M	20 K
COCA	440 M	2 M
Google n -gram	1 T	13 M

This means vocabulary size grows sublinearly with corpus size.

Common Normalization Techniques

- **Case Folding:** converts all text to lowercase. *The* $\rightarrow$ *the*
- **Lemmatization:** converts words to their base form (lemma) using linguistic rules. Example: *is* $\rightarrow$ *be*, *stories* $\rightarrow$ *story*
He is reading sci-fi stories *becomes* He be read sci-fi story.
- **Stemming:** a simpler, rule-based method that chops off suffixes (e.g., Porter Stemmer). Example: *accurate* $\rightarrow$ *accur*

We still have a big problem left, which are typos.

Levenshtein edit distance for spelling correction measures the similarity between two strings by calculating the **minimum number of edit operations** needed to transform one into the other.

Edit Operations (with Costs):

- Deletion: 1
- Insertion: 1
- Substitution: 2 (treated as a deletion + insertion)

```
intention
*execution
dss is

def min_edit_dist(source, target):
    n, m = len(source), len(target)
    D[n+1,m+1] = zeroes

    for i in range(1, n):
        D[i, 0] = D[i-1, 0] + del_cost(source[i])
    for j in range(1, m):
        D[0, j] = D[0, j-1] + ins_cost(target[j])

    for i in range(1, n):
        for j in range(1, m):
            d[i, j] = min(D[i-1, j] + del_cost(source[i]),
                          D[i-1, j-1] + sub_cost(source[i], target[j]),
                          D[i, j-1] + ins_cost(target[j]))
    return D
```

We need 8 operations to go from intention to execution (3 substitutions, a deletion and an insertion).

The algorithm explores all possible edits and finds the minimal-cost path through a matrix representation of the two strings.

For example, the least effective way would be to delete the whole string 'intention' and inserting the whole string execution. This works, but it doesn't use overlaps that instead are indeed present.

The result: **A matrix D** where $D[i][j]$ is the minimum cost to transform the first i characters of the source into the first j characters of the target.

We create an $(n + 1) \times (m + 1)$ matrix (rows = source, columns = target).

Example: Source = "intention" (9 letters); Target = "execution" (9 letters) $\rightarrow$ Matrix size = 10×10


```

for i in range(1, n):
    D[i, 0] = D[i-1, 0] + del_cost(source[i])
for j in range(1, m):
    D[0, j] = D[0, j-1] + ins_cost(target[j])

```

After initializing the matrix D , we then initialize the first row and column this way. In fact, converting a word to an empty string has the cost of deleting all characters (so cost = i). Viceversa, converting an empty string to a word has the cost of inserting all characters (cost = j). So the top row and left column are filled with incremental numbers: 0, 1, 2, 3, ...

```

for i in range(1, n):
    for j in range(1, m):
        D[i, j] = min(
            D[i-1, j] + del_cost(source[i]),           # deletion
            D[i-1, j-1] + sub_cost(source[i], target[j]), # substitution
            D[i, j-1] + ins_cost(target[j])           # insertion
        )

```

At each cell (i, j) :

- We look *one step back* in each direction (up, left, diagonal).
- We add the corresponding cost for that operation.
- We take the **minimum** of those three options — the cheapest way to get there.

So:

- $D[i-1, j] + \text{del_cost}$ → delete a char from source
- $D[i, j-1] + \text{ins_cost}$ → insert a char to match target
- $D[i-1, j-1] + \text{sub_cost}$ → substitute if chars differ, or 0 if same

To calculate the distance between two strings, we start with an 'empty string' $\#$ and compute $D(i, j)$ for all i ($0 < i < n$) and j ($0 < j < m$), given the following—according to Levenshtein distance—operations:

$$\min \begin{cases} D(i-1, j) + 1 & \text{(deletion from X)} \\ D(i, j-1) + 1 & \text{(insertion into X)} \\ D(i-1, j-1) + \begin{cases} 2, & \text{if } X(i) \neq Y(j) \text{ (substitution)} \\ 0, & \text{if } X(i) = Y(j) \text{ (match, no cost)} \end{cases} \end{cases}$$

Assume we have the following two strings: **abcd** and **aced**.

We put the source in vertical and the target in horizontal. Each preceded by the empty string $\#$. We first fill the left column and the bottom row. To simply put it, if we want to go from the empty string to the empty string, cost is 0. So $D[\#, \#]=0$. Then, if we want to go from the string 'a' to the string $\#$, cost is 1: $D[a, \#]=1$. Then, from 'ab' to $\#$, cost is 2. Etc. Same reasoning for the bottom row: e.g., $D[\#, b]=2$ because to go from 'ab' to ' $\#$ ' it costs two (two delete operations).

We get:

d	4	3	2	3	2
c	3	2	1	2	3
b	2	1	2	3	4
a	1	0	1	2	3
#	0	1	2	3	4
	#	a	c	e	d
		^1st	^2nd	^3d	^4th

I have annotated the steps that we followed. We start with the first $\#$ rows (bold), those always range from empty string to the length of the string. Then, we can start from the bottom of the 1st column (provided that our table lay-out is similar), from the cell that is not bold. There are three numbers around it (0 , 1  and 1 ). We first check : if the strings are the same, then we have a cost of 0, if they are not, then using the diagonal character costs 2 (substitution). If we use , or , the cost is 1. We have to **add** this cost to whatever is already in those respective cells (so 0 + either 0 (if similar char) or 2 (if not similar), 1 + 1 and 1 + 1). Then we select what the minimum original value

+ cost is as the current value. For this cell, that is zero (because similar characters). This operation we continue on applying. For the next cells upwards, nothing exiting happens (just adding ones). Then on to the second column, above the 2. We have ↖ 1 (+ 2 because not similar), ← 0 (+ 1) and ↓ 2 (+ 1). The minimum value of these is 0 + 1 = 1. etc etc.

Finally, if we trace the shortest path back from the top right corner, we get the operations we need to do. If we go ←, that means an addition, if we go ↓, that means a deletion, if we go ↖, it's either substitution or nothing, depending on the value. So: 2 → 2 (nothing, both d), 2 → 1 (insert e), 1 → 1 (nothing, both c), 1 → 0 (delete b), 0 → 0 (do nothing, both a).

Modern Data-Driven Encodings (Sub-Word Methods)

Modern tokenization methods break words into smaller, reusable units to handle unseen words and multilingual text efficiently.

- **Byte-Pair Encoding (BPE):** starts with characters, merges the most frequent adjacent pairs, and repeats until reaching the desired vocabulary size.
- **WordPiece Encoding:** Similar to BPE but merges based on **probability**—two pieces are combined if their joint probability is higher than their individual ones. Produces compact, language-agnostic subword vocabularies used in models like BERT.

Classification

N-grams Language Models

A **Language Model (LM)** assigns probabilities to sequences of words. It answers questions like:

- *What is the probability of the next word given the previous ones?*
e.g. $P(\text{bike} \mid \text{I lost my balance and fell off my})$
- *What is the probability of an entire sentence?*
e.g. $P(\text{I am Sam})$

Formally, the probability of a sequence is:

$$P(\text{bike} \mid \text{I lost my balance and fell off my}) = \frac{C(\text{I lost my balance and fell off my bike})}{C(\text{I lost my balance and fell off my})}$$

$$P(w_{1:n}) = P(w_1) P(w_2 \mid w_1) P(w_3 \mid w_{1:2}) \dots P(w_n \mid w_{1:n-1})$$

$$= \prod_{k=1}^n P(w_k \mid w_{1:k-1})$$

$$P(w_{1:n}) = \prod_{k=1}^n P(w_k \mid w_{1:k-1})$$

Why probabilities?

Counting-based probabilities help systems *predict* and *rank* possible continuations or translations. This is why LMs are used for autocorrect, MT, speech recognition, etc.

Classifiers vs Language Models

Classifiers use labels to predict the data, LMs use the data to predict the data (selfsupervised).

Predictors all require representation methods to convert text into a computation space.

Tokenization is (once again) very important, because better representations imply less noise, and therefore better models. Data preparation is where most errors introduce themselves.

Maximum Likelihood Estimation (MLE)

We would ideally compute: $P(w_k \mid w_{1:k-1}) \dots$ but real contexts are long and often **never occur** in the training data.

To make estimation feasible, we assume the **Markov assumption**:

┃ A word only depends on the previous (n-1) words, not the entire history.

Examples:

- Unigram: $P(w_k)$
- Bigram: $P(w_k \mid w_{k-1})$

- Trigram: $P(w_k | w_{k-2}, w_{k-1})$

With bigrams, we approximate:

$$P(w_n | w_{n-1}) \approx \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

Where:

- $C(w_{n-1})$ = count of the previous word
- $C(w_{n-1}, w_n)$ = count of the two words appearing consecutively (a **bigram**)

Training sentences:

1. `<s> I am Sam </s>`
2. `<s> Sam I am </s>`
3. `<s> I do not like green eggs and ham </s>`

From these, we can compute bigram probabilities:

- $P(I | < s >) = 2/3 = 0.67$ (`<s> I` appears twice; `<s>` appears three times total)
- $P(Sam | am) = 1/2 = 0.5$
- $P(do | I) = 1/3 = 0.33$
- $P(am | I) = 2/3 = 0.66$

Sequence probability example

Using bigrams, the probability of the whole sequence is the product of each conditional probability:

$$P(w_{1:n}) = \prod_{k=1}^n P(w_k | w_{k-1})$$

For a short sentence like `<s> I am Sam </s>` you multiply:

- $P(I | < s >)$
- $P(am | I)$
- $P(Sam | am)$
- $P(< /s > | Sam)$

This multiplication gives the probability of the full sentence.

Why log probabilities

Multiplying many probabilities (often < 0.01) quickly underflows to zero.

Instead, we work with **log probabilities**:

$$\log(P_1 P_2 P_3) = \log P_1 + \log P_2 + \log P_3$$

This prevents numerical issues and simplifies computation.

Evaluating LMs: Perplexity

We aren't classifying or ranking anything... so how do we evaluate probability models?

Perplexity measures how well a model predicts a test set:

$$PP(W) = P(w_{1:N})^{-\frac{1}{N}} = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

Interpretation:

- Low perplexity $\rightarrow$ model assigns **high probability** to the actual data (good!)

- High perplexity → model is “confused” by the data

Imagine we have a test sentence: **"I want to eat..."**

The next actual word in the test set is **"food"**.

A model which assigns a probability of 0.5 to food, has $\frac{1}{0.5} = 2$, which is definitely higher than a random model which assigns to all words, e.g. 0.01 ($\frac{1}{0.01} = 100$).

Then, the productory is to consider all the N words predicted. Each of them is assigned a probability based solely on the previous word, here.

If you did **not** take the N-th root, perplexity would get exponentially worse as the sequence gets longer. A 1000-word text would always look “worse” than a 10-word text purely because there are more words. Taking the N-th root normalizes the probability across sequence length so:

- 10-word test sets
- 1000-word test sets

...are directly comparable.

Also, as we have seen, multiplying many tiny probabilities: $P(W) = p_1 p_2 p_3 \dots p_N$ causes **underflow** (numbers become too small to represent). So we compute: $\log P(W) = \log p_1 + \log p_2 + \log p_3 + \dots$

This is much safer because logs turn multiplication into addition.

But perplexity needs the probability, not the log probability

Perplexity formula uses: $PP(W) = P(W)^{-1/N}$. But we’ve only computed **log P(W)**. So we get back P(W) by exponentiating: $P(W) = \exp(\log P(W))$

Therefore:

$$PP(W) = \exp\left(-\frac{1}{N} \log P(W)\right) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log p_i\right)$$

Handling Zeros & Smoothing

Two big problems occur naturally:

1. OOV (Out of Vocabulary) words

The test set may contain words never seen in training.

Solution: Replace low-frequency words with **<UNK>** in training.

2. Unseen sequences

If a bigram never occurs, its probability is **0**, and a single zero wipes out the probability of the entire sentence.

Smoothing redistributes probability mass to unseen events.

Methods:

- **Laplace (Add-1):** $P_{\text{Laplace}}(w_n | w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + V}$

Simple, but tends to over-smooth.

- **Add-k:** Same idea, but add a small fractional amount instead of 1. K is inferred from the data.
- **Backoff:** If trigram count = 0 → try bigram → try unigram.
- **Interpolation:** Combine unigram + bigram + trigram probabilities using learned weights.
- **Kneser–Ney:** Advanced method which focuses on how many *different contexts* a word appears in. Some words occur very frequently but only after *one* specific word.

Examples:

- “Francisco” mainly follows “San”
- “Ville” mainly follows “Louis”
- “Jr.” mainly follows people’s names
- “%” mainly follows numbers
- “of” occurs after almost everything

If a word has **high frequency but low context diversity**, then its probability should be **low** when backing off to unigrams — because it's not a generally likely word; it's tied to a specific context.

Conversely, a word like “**the**” appears after *many* different words, so it should have a **high** probability when backing off.

Naive Bayes Classification

A generative classifier based on a strong assumption: we assume all features contribute independently to the classification, regardless of correlations between the features.

Even though this assumption is false (words are dependent!), Naive Bayes works surprisingly well for text.

Also, it is *naive* in the sense that the probability of a label given a sentence is independent of the order in which the words appear in the sentence; this does not reflect what is true about language (grammar, syntax, ...). Here we treat language just as a bag of words and each sentence is a handful of those. The score is just a multiplication of probabilities $P(\text{word}|\mathbf{y})$, as you can see in the formula:

$$\mathbf{x} = \langle \text{great}, \text{movie} \rangle$$

$$P(\text{POS} \mid x_1, x_2) = P(x_1 \mid \text{POS}) P(x_2 \mid \text{POS}) P(\text{POS})$$

$$P(\text{NEG} \mid x_1, x_2) = P(x_1 \mid \text{NEG}) P(x_2 \mid \text{NEG}) P(\text{NEG})$$

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^J P(x_j \mid y)$$

Our sentence contains the words great and movie.

Obviously for Bayes' theorem we should also divide by $P(x_1) \cdot P(x_2)$, but since this is the same for both $P(\text{POS}|\cdot)$ and $P(\text{NEG}|\cdot)$, and we don't actually care about true probabilities, but just about seeing which is bigger, we can omit that.

Formula:

$$\hat{y} = \arg \max_y P(y) \prod_j P(x_j | y)$$

Where:

- $P(y)$ = prior probability of class
- $P(x_j | y)$ = probability of word x_j under class y

$P(x_j | y)$ implies that for x_j , we get its conditional probability given every value of y , so 'how much does feature value x_j occur under (for example) positive labels, and under negative labels'.

There are two ways to compute $P(x_j | y)$:

- **Multinomial NB:** $P(t \mid c)$ estimated from token counts (total number of occurrences of term t in class c). Add-one (Laplace) smoothing adds 1 to every term count and adds $|V|$ to the denominator. $\hat{P}(x_j \mid y) = \frac{C(x_j, y)}{\sum_{x_i \in V} C(x_i, y)}$
The multinomial model counts **how many times** each word appears in all documents of class y .
 - $C(x_j, y)$ = total token count of word x_j in all documents of class y
 - denominator = total number of word tokens in class y
 - Typically smoothed with Laplace: add 1 to numerator, add $|V|$ to denominator.
- **Bernoulli NB:** $P(t \mid c)$ estimated from *document-level presence* (fraction of documents in class c that contain t). Laplace here adds 1 to numerator and 2 to denominator (one pseudo-count for presence and one for absence). The Bernoulli classifier also explicitly uses $1 - P(t \mid c)$ for absent terms.

Let:

- N_y = number of documents with label y
- $N_y(x_j)$ = number of documents with label y that contain word x_j

Then the Bernoulli MLE is: $\hat{P}(x_j \mid y) = \frac{N_y(x_j)}{N_y}$

This is the **Bernoulli analogue** of your multinomial formula.

In these notes we will use Multinomial, in task 9 in the exercises we will use Bernoulli.

$x_1 = \text{bad}$	$x_2 = \text{great}$	$x_3 = \text{movie}$	y
0	1	1	👍
0	1	1	👍
0	0	1	👍
1	0	1	👎
1	0	0	👎

$$P(y | x_1, x_2, x_3) = P(x_1 | y) P(x_2 | y) P(x_3 | y) P(y)$$

$P(\text{bad} \text{👍}) = 0 / 3$	$P(\text{bad} \text{👎}) = 2 / 2$
$P(\text{great} \text{👍}) = 2 / 3$	$P(\text{great} \text{👎}) = 0 / 2$
$P(\text{movie} \text{👍}) = 3 / 3$	$P(\text{movie} \text{👎}) = 1 / 2$
$P(\text{👍}) = 3 / 5$	$P(\text{👎}) = 2 / 5$

$D = \text{"bad movie"}$	
$P(\text{👍} D) = P(x_1) * P(x_3) * P(\text{👍}) = 0 / 3 * 3 / 3 * 3 / 5 = 0.00$	
$P(\text{👎} D) = P(x_1) * P(x_3) * P(\text{👎}) = 2 / 2 * 1 / 2 * 2 / 5 = 0.20$	

This example is actually using Bernoulli NB, not multinomial NB... As you can see, the denominator divides by 3 and 2, instead of 5 and 3. And it is not smoothing.

Now we'll use multinomial.

Based on the equation from the book (*this is indeed using multinomial*):

$$\hat{P}(x_j | y) = \frac{C(x_j, y)}{\sum_{x_i \in V} C(x_i, y)}, \quad (4)$$

where C is a co-occurrence count of the two inputs, and V is our vocabulary. We then multiply and sum these (product $\prod$), and multiply them by prior $P(y)$ — i.e., the probabilities of a value in y occurring.

The algorithm follows the following steps:

- Given: a smoothing factor ϵ (there are more hyperparameters such as the assumed distribution, but we don't discuss those):
- At training (fitting) time, given: a dataset X , we do the following:
 - We get all probabilities (right-hand part after argmax) for our current data, and store it.
 - Any C , we optionally smooth with ϵ (typically +1). For the denominator this means that: $\sum_{x_i \in V} (C(x_i, y) + \epsilon)$, which is equal to $(\sum_{x_i \in V} C(x_i, y)) + |V| \cdot \epsilon$
- At test (inference) time, given new instance t , we:
 - Look up the pre-calculated probabilities for the features present in $t(x_j)$, with respect to each possible label in y .
 - We return label with the highest probability ($\arg \max_y$) as $\hat{y}$.

To put it simply, we have:

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^J P(x_j | y). \quad (5)$$

$P(y)$ is obvious.

Now, for each of these y , we have to compute $\prod_{j=1}^J P(x_j | y)$, i.e. for each feature (all, great, not, bad...) we have to see the probability of that feature given the label y . Simply, we count how many times feature x_j appears when the label is y , and we divide by the total number of words where label is y .

However, since we don't want that probability to be 0, because otherwise we would have that productory all null, we add a smoothing of +1 to the numerator, and is $1 * \{\text{number of features}\}$ to the denominator, since it has to normalize these new values, to which we added 1. E.g., $p(\text{all} | +) = (2 + 1) / (9 + 4)$. 9 is the total number of times that a feature is 1 under $y=1$.

The algorithm follows the following steps:

- Given: a smoothing factor ϵ (there are more hyperparameters such as the assumed distribution, but we don't discuss those):
- At training (fitting) time, given: a dataset X , we do the following:
 - We get all probabilities (right-hand part after argmax) for our current data, and store it.
 - Any C , we optionally smooth with ϵ (typically +1). For the denominator this means that: $\sum_{x_i \in V} (C(x_i, y) + \epsilon)$, which is equal to $(\sum_{x_i \in V} C(x_i, y)) + |V| \cdot \epsilon$

- At test (inference) time, given new instance $\mathbf{t}$, we:
 - Look up the pre-calculated probabilities for the features present in $\mathbf{t}$ (x_j), with respect to each possible label in y .
 - We return label with the highest probability ($\arg \max_y$) as $\hat{y}$.

To put it simply, we have:

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^J P(x_j | y). \quad (6)$$

$P(y)$, i.e. the prior of all the possible y values, is obvious.

Now, for each of these y , we have to compute $\prod_{j=1}^J P(x_j | y)$, i.e. for each feature (all, great, not, bad...) we have to see the probability of that feature given the label y . Simply, we count how many times feature x_j appears when the label is y , and we divide by the total number of times any x_j is 1 under y . $\hat{P}(x_j | y) = \frac{C(x_j, y)}{\sum_{x_i \in V} C(x_i, y)}$,

However, since we don't want that probability to be 0, because otherwise we would have that productory all null, we add a smoothing of +1 to the numerator, and is $1 * \{\text{number of features}\}$ to the denominator, since it has to normalize these new values, to which we added 1. E.g., $p(\mathbf{all} | +) = (2 + 1) / (9 + 4)$

9 is the total number of times that a feature is 1 under $y=1$.

Task 8

- Given the dataset below, and the following instance: $d = \mathbf{all \ was \ not \ great}$ — calculate $\hat{y}$, counts are smoothed by +1.

all	great	not	bad	y
0	0	0	1	—
0	1	0	0	—
0	0	0	1	—
1	0	0	1	—
1	1	1	1	+
0	0	1	1	+
1	1	1	0	+

▼ Solution (Full)

The counts are obtained by looking at a particular word, and summing their frequencies across **one** particular label: e.g., **all** occurs two times under $y = +$ (2, and we add 1 for Laplace smoothing). Note that **not** never occurs under $y = -$, so $p(\mathbf{not} | -) = 0 + 1$ — again, the +1 being Laplace smoothing (always one for the numerator). The denominator is smoothed as (sum of all frequencies under a label + $|V| \cdot \epsilon$, where ϵ is the smoothing factor, i.e., +1) — also see [here](#). So for +, we sum all the frequencies:

$$\sum \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \begin{matrix} + \\ + \\ + \end{matrix} = 9$$

Then $9 + |V| \cdot \epsilon = (9 + (4 \cdot 1))$ (we drop the $\cdot 1$).

“was” is not in the vocabulary → ignored.

Features present (in V : $\{\mathbf{all}, \mathbf{not}, \mathbf{great}\}$).

This gives for all probabilities:

$$\begin{aligned} p(\mathbf{all} | +) &= (2 + 1) / (9 + 4) \approx .213 & p(\mathbf{all} | -) &= (1 + 1) / (5 + 4) \approx 0.222 \\ p(\mathbf{great} | +) &= (2 + 1) / (9 + 4) \approx .213 & p(\mathbf{great} | -) &= (1 + 1) / (5 + 4) \approx 0.222 \\ p(\mathbf{not} | +) &= (3 + 1) / (9 + 4) \approx .308 & p(\mathbf{not} | -) &= (0 + 1) / (5 + 4) \approx 0.111 \\ p(\mathbf{bad} | +) &= (2 + 1) / (9 + 4) \approx .213 & p(\mathbf{bad} | -) &= (3 + 1) / (5 + 4) \approx 0.444 \\ p(\mathbf{yes}) &= 3/7 \approx 0.429 & p(\mathbf{no}) &= 4/7 \approx 0.571 \end{aligned}$$

$$\begin{aligned} &\mathbf{all} \ * \ \mathbf{great} \ * \ \mathbf{not} \ * \ \text{prior} \\ p(-|x) &= 0.222 * 0.222 * 0.111 * 0.571 \approx .003 \\ p(+|x) &= 0.213 * 0.213 * 0.308 * 0.429 \approx .006 \\ \hat{y} &= \arg \max_y [.003, .006] = 1 \quad (\text{i.e., } +) \end{aligned}$$



You can check if the nominators are correct by summing them for a particular label (vertically per column), this should give the denominator for all columns (e.g.,: $2 + 2 + 3 + 2 = 9$ and $1 + 1 + 1 + 1 = 4$).

Decision Trees (ID3)

Decision Trees learn rules like:

```
if 'vision' in document:
    class = 'CV'
elif 'language' in document:
    class = 'NLP'
else:
    class = 'TM'
```

The model **infers these rules** automatically.

The key concepts:

- **Entropy** (impurity measure)

In a decision tree, entropy is a **measure of impurity or uncertainty within a dataset**. It's used to determine the best way to split the data at each node to create the purest possible subsets, thereby reducing disorder and increasing certainty. The algorithm calculates the entropy for each possible split and chooses the one that results in the largest reduction in entropy (known as information gain). A node with 100% of instances belonging to one class has an entropy of 0, while a node with an equal mix of classes has the maximum entropy of 1.

- **Information Gain** (entropy reduction when splitting on a feature)

ID3 repeatedly:

1. Computes entropy of the current set
2. Computes entropy of splitting on each feature (e.g., “does the word ‘free’ appear?”)
3. Chooses the split with highest Information Gain
4. Recurses on subsets
5. Stops when a subset is pure or no features remain

This produces an interpretable ruleset, but can overfit without pruning.

Decision trees algorithms recursively make splits of the data, until they hit some criterion. Through this, they produce 'rules', which, at the end of the tree, we predict the majority label for the leftovers. In the lecture, we restrict ourselves to a *single* split of the data, and looked at Information Gain:

where Entropy can be calculated using:

Note that:

$$p_i = \frac{C(S_i)}{C(S)}$$

Here:

- Y = The target column (label) we related the split quality to.
- x_n = The feature (word / token / etc) we are testing our split on.
- v = Each unique value of x , so if $x = [0, 1, 2, 1, 1, 2, 3, 1]$, then v might be 0, 1, 2, or 3.
- Y_v = The subset of v with respect to Y , so if $y = [0, 1, 0, 1]$ and $x = [0, 1, 1, 0]$, then $Y_{v=1}$ gives us $y = [1, 0]$ and $x = [1, 1]$.
- $|Y|$ and $|Y_v|$ = the sizes of the respective splits.
- p_i is the probability of drawing label value i from S

free	money	account	transfer	y	
1	1	0	0	spam	
0	1	1	1	ham	
1	0	1	0	ham	
1	0	1	1	spam	
0	1	0	1	spam	
1	0	0	1	spam	
0	0	1	0	ham	
0	1	1	0	ham	
0	1	0	0	spam	

$E(C) = - \sum_{i=1}^n P(c_i) \log_2 P(c_i)$
 $E(\text{ham}) = - \left(\frac{4}{9} \cdot \log_2 \frac{4}{9} \right)$
 $E(\text{spam}) = - \left(\frac{5}{9} \cdot \log_2 \frac{5}{9} \right)$
 $E(Y) = E(\text{ham}) + E(\text{spam})$
 $E(Y) = -(-0.52 + -0.47)$
 $= 0.991$

E = Entropy, *P* = proportion / probability, *C* = class,
IG = Information Gain

Entropy of the full dataset (before splitting).
 There's a typo. E(Y) = E(ham)+E(spam)

		spam	ham	total
free	0	2	3	5
	1	3	1	4

$$E(\text{free}, Y) = \sum_{c \in Y} P(c) E(c)$$

$$E(\text{free}=0, Y) = - (P(\text{free}=0, \text{spam}) \cdot \log_2 P(\text{free}=0, \text{spam})) - (P(\text{free}=0, \text{ham}) \cdot \log_2 P(\text{free}=0, \text{ham}))$$

$$E(\text{free}=1, Y) = - (P(\text{free}=1, \text{spam}) \cdot \log_2 P(\text{free}=1, \text{spam})) - (P(\text{free}=1, \text{ham}) \cdot \log_2 P(\text{free}=1, \text{ham}))$$

$$E(\text{free}, Y) = P(\text{free}=0) \cdot E(\text{free}=0, Y) + P(\text{free}=1) \cdot E(\text{free}=1, Y)$$

Let's try to see how much information gain we obtain if splitting on free.

		spam	ham	total
free	0	1	3	4
	1	4	1	5

$$E(\text{free}, Y) = \sum_{c \in Y} P(c) E(c)$$

$$E(\text{free}=0, Y) = - \left(\frac{1}{4} \cdot \log_2 \frac{1}{4} \right) - \left(\frac{3}{4} \cdot \log_2 \frac{3}{4} \right)$$

$$E(\text{free}=1, Y) = - \left(\frac{4}{5} \cdot \log_2 \frac{4}{5} \right) - \left(\frac{1}{5} \cdot \log_2 \frac{1}{5} \right)$$

$$E(\text{free}, Y) = \frac{4}{9} \cdot 0.811 + \frac{5}{9} \cdot 0.722 = 0.762$$

$$IG = E(Y) - E(\text{free}, Y) = 0.991 - 0.762 = 0.229$$

When computing E(free,Y), we weigh each subset's entropy by the proportion of samples in that subset.

- The dataset started with entropy = **0.991** (quite mixed, almost 50/50 spam/ham).
- If we split on **“free”**, the entropy becomes **0.762** (more pure).
- The split reduces uncertainty by **0.229**.

ID3 will repeat this process for **every feature** (words like “account”, “transfer”, “money”...).

The feature with the **highest Information Gain** is chosen as the first node in the tree.

Regression Models (Linear & Logistic)

Linear Regression fits a line:

$$Y = \beta_0 + \beta_1 X$$

The coefficients are computed using covariance and variance formulas.

city	students (X)	alcohol (Y)	$(X - \bar{X})^2$	$(X - \bar{X}) \cdot (Y - \bar{Y})$
Tilburg	26	41	$(26 - 18)^2 = 64$	$(26 - 18) \cdot (41 - 29) = 96$
Eindhoven	21	37	$(21 - 18)^2 = 9$	$(21 - 18) \cdot (37 - 29) = 24$
Wageningen	6	9	$(06 - 18)^2 = 144$	$(06 - 18) \cdot (09 - 29) = 240$
Σ	53	87	217	360

- $\bar{X} = 53/3 \approx 18, \bar{Y} = 87/3 = 29$
- $\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{360}{217} \approx 1.66$
- $\beta_0 = \bar{Y} - \beta_1 \cdot \bar{X} = 29 - 1.66 \cdot 18 = -0.88$
- $\hat{y} = \beta_0 + \beta_1 \cdot X = -0.88 + 1.66 \cdot X$

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2}{N}}$$

$$= \sqrt{\frac{(42.28 - 41)^2 + (33.98 - 37)^2 + (9.08 - 9)^2}{3}} = 1.89$$

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} = 1 - \frac{10,7652}{608} = .982$$

Evaluation:

$$MAE = \frac{\sum_{i=1}^N |\hat{y}_i - y_i|}{N}$$

$$MSE = \frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2}{N}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2}{N}}$$

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

- Given $y = [8, 3, 1]$ and $\hat{y} = [4, 3, 2]$, calculate MAE, MSE, RMSE, and R^2 .

▼ Solution (Full)

$$MAE = \frac{|4 - 8| + |3 - 3| + |2 - 1|}{3} = \frac{4 + 0 + 1}{3} \approx 1.667$$

$$MSE = \frac{(4 - 8)^2 + (3 - 3)^2 + (2 - 1)^2}{3} \approx 5.667$$

$$RMSE = \sqrt{\frac{(4 - 8)^2 + (3 - 3)^2 + (2 - 1)^2}{3}} \approx 2.38$$

$$R^2 = 1 - \frac{(8 - 4)^2 + (3 - 3)^2 + (1 - 2)^2}{(8 - 4)^2 + (3 - 3)^2 + (1 - 4)^2} = 1 - \frac{17}{26} \approx 0.346$$

* $\bar{y} = (8 + 3 + 1)/3 = 4$

Logistic Regression is used for binary classification.

It computes a linear score ($z = \beta_0 + \beta_1 \times x$) and then applies a sigmoid to convert it to a probability:

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

Weights are learned via **Gradient Descent**.

Extremely effective for high-dimensional features like TF-IDF vectors.

Support Vector Machines (SVM)

SVMs aim to find a decision boundary that:

1. Separates classes

2. Has the maximum margin to the closest points (support vectors)

Intuitively:

- A larger margin → better generalization
- Only a few critical points (“support vectors”) determine the boundary

The **kernel trick** lets SVMs handle non-linear patterns by implicitly projecting data into higher-dimensional spaces (e.g., RBF kernel).

k-Nearest Neighbors (k-NN)

k-NN stores all training samples and classifies new samples based on similarity.

Process:

1. Store all training vectors
2. For a test vector, compute distance to all training vectors
3. Select the k closest neighbors
4. Assign the majority label

Common distances:

- Euclidean Distance: $\sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- Manhattan Distance: $\sum_{i=1}^n |x_i - y_i|$
- Minkowski Distance: $(\sum_{i=1}^n |x_i - y_i|^p)^{\frac{1}{p}}$
- Cosine Distance: $1 - \frac{\vec{p} \bullet \vec{q}}{\sqrt{\vec{p} \bullet \vec{p}} \cdot \sqrt{\vec{q} \bullet \vec{q}}}$

You can weight neighbors by distance (closer neighbors count more) or by label rarity (Inverse Document Frequency).

Generalization & Evaluation

A model should perform well on **unseen** data.

Underfitting:

- Model too simple
- High error on both train and test sets

Overfitting:

- Model memorizes training data
- Low train error, high test error
- Caused by: small k, deep trees, too many features, unregularized models

Hyperparameter tuning must be done on a **validation set**, not the test set.

Examples: k (in k-NN), learning rate, tree depth, regularization strength, etc.

Evaluation schemes:

- Hold-out split
- k-fold cross-validation
- Leave-one-out (extreme case of k-fold)
- Stratified splits (preserve class ratios)

These prevent models from fooling you into thinking they are better than they really are.

Representation

- We looked at some interesting language features.
- We discussed how to convert text into vectors, using a discrete n-gram representation.

- Using this vector space, we looked at distance and how it might be employed for similarity.
- We also looked at how we might infer probabilistic models from data, both to model, and predict from, the data.

This week marks the shift from **discrete, sparse representations** (like Bag-of-Words) to **dense, distributed representations** (embeddings). We explore how to model meaning using counts, predictions, and neural networks. The big picture: stop hand-crafting features, and let models *learn* them.

Meaning as Counts: Distributional Semantics

The Nature of Language Data

Language is symbolic and categorical—we represent things like *trees* with discrete tokens like “tree.” Unlike Bag-of-Words, which ignores order, real language meaning depends on context and sequence. One-hot vectors, though simple, are extremely sparse and provide no built-in notion of similarity (e.g., “cat” and “dog” look unrelated).

Distributional Similarity (and PPMI)

Firth’s classic idea: “*You shall know a word by the company it keeps.*”

Words that appear in similar contexts tend to have similar meanings. To capture this, we build **word-context matrices** where rows are words, columns are context words, and entries count how often they appear near each other.

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Weighting With PPMI

Raw co-occurrence counts overweight frequent words like “the”.

Positive Pointwise Mutual Information (PPMI) reweights these counts by comparing observed co-occurrence frequency to what would happen by chance. Negative PMI values (less than chance) are set to zero for stability. A common tweak is to smooth the context distribution to avoid extreme PMI spikes caused by rare words.

$$PMI(w, c) = \log_2 \frac{P(w, c)}{P(w)P(c)}$$

- $P(w, c)$ is the observed probability that w and c actually occur together in the corpus within the defined context window.
- $P(w)P(c)$ is the probability that w and c would occur together **if they were entirely independent** (i.e., purely by chance). This is calculated by multiplying their individual marginal probabilities.

If the probability of co-occurrence is lower than chance (i.e. if numerator < denominator), then the logarithm is negative. In this case we take 0.

$$PPMI(w, c) = \max(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0)$$

All the probabilities are calculated dividing by the summation of occurrences of all context words AND words. For example:

$\downarrow w, c \rightarrow$	aardvark	baboon	cat	dolphin	kākāpō	Σ
air	0	3	1	6	3	13
ball	2	0	1	1	1	5
breathes	1	0	1	27	2	31
claws	664	12	1211	5	305	2197
water	0	4	1	14	0	19
Σ	667	19	1215	53	311	2265

- Using the co-occurrence matrix below, calculate PPMI(claws, cat).

$$PPMI(\text{claws}, \text{cat}) = \max\left(\log_2 \frac{P(w, c)}{P(w)P(c)}, 0\right) = \log_2 \left(\frac{1211/2265}{2197/2265 \cdot 1215/2265}\right) = 0.039$$

An other example:

$$\begin{aligned}
 P(w = \text{information}, c = \text{data}) &= \frac{3982}{11716} = .3399 \\
 P(w = \text{information}) &= \frac{7703}{11716} = .6575 \\
 P(c = \text{data}) &= \frac{5673}{11716} = .4842 \\
 \text{ppmi}(\text{information}, \text{data}) &= \log_2 \frac{.3399}{.6575 \times .4842} = .0944
 \end{aligned}$$

A known issue is that PPMI tends to over-emphasize the relevance of **very rare words** because their $P(w)$ and $P(c)$ terms in the denominator are minuscule, leading to an extremely high ratio.

To address this, the context probability $P(c)$ can be modified (or "smoothed") using a weighting factor α (typically $\alpha = 0.75$):

$$P_\alpha(c) = \frac{\text{count}(c)^\alpha}{\sum_c \text{count}(c)^\alpha}$$

The modified PPMI formula then becomes:

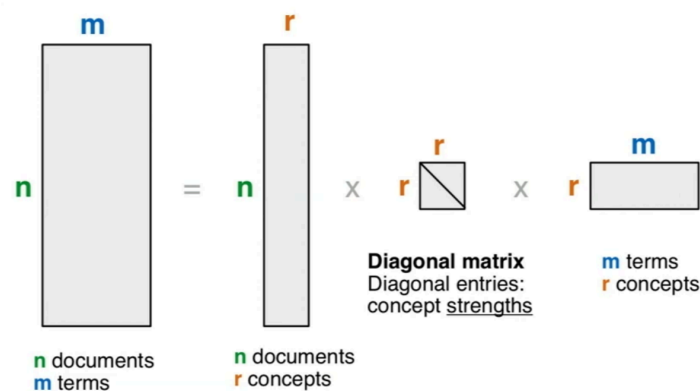
$$PPMI_\alpha(w, c) = \max(\log_2 \frac{P(w, c)}{P(w)P_\alpha(c)}, 0)$$

Dimensionality Reduction with LSA / SVD

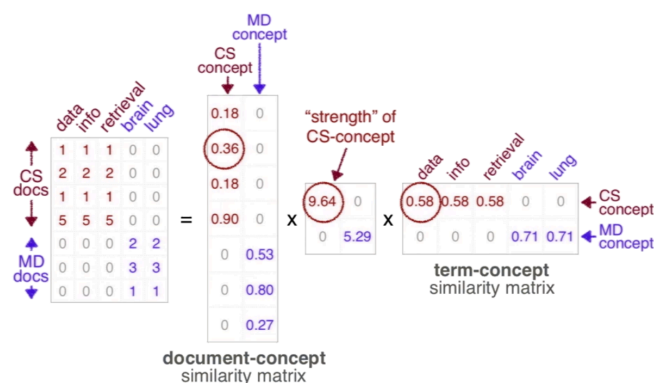
Word-context matrices are huge ($|V|$) and sparse, so we compress them using **Singular Value Decomposition (SVD)**, since dense vectors work much better for most language problems.

In NLP, SVD is 'disguised' as Latent Semantic Analysis (LSA).

This gives us lower-dimensional **dense vectors** that capture latent semantic structure (e.g., ~300 dimensions instead of tens of thousands).



In this illustrative example, we apply it to a document-term matrix, but it also works for term context matrices.



Matrix Component	Represents	Interpretation
Document-Concept Matrix	U in $X = U\Sigma V^T$	How much each document relates to the learned concepts (e.g., CS or MD).
Diagonal Matrix	Σ	The "strength" of each concept. This is where the dimensionality reduction occurs.
Term-Concept Matrix	V^T in $X = U\Sigma V^T$	How much each word relates to the learned concepts. Words that relate strongly to the same concepts are considered similar (e.g., "data," "info," and "retrieval" all map to the "CS concept").

Truncating SVD to the top components removes noise and highlights the most important semantic factors. In fact, the diagonal values of the matrix Σ encode most of the informative content of the matrix X .

Predicting Meaning: Neural Word Embeddings

In traditional count-based models, meaning is **atomic**. Each unique word is treated as an isolated, independent unit.

- **Representation:** A word's meaning is represented by a single index in a massive, sparse vector, often symbolized by its capitalized form (e.g., the meaning of 'life' is simply **LIFE**).
- **Limitation:** This system cannot inherently capture any relationship between words; "cat" is as far from "dog" as it is from "airplane" because their corresponding one-hot vectors have a dot product of zero.

We want to have a broader sense of meaning:

- cat is similar to dog (**categories**)
- cold is opposite of hot (**antonyms**)
- fake/copy **different connotations**
- capture related **sequences**: buy → sell → pay
- mouse = rodent and controller (**polysemy**)
- couch = sofa (**synonymy**)

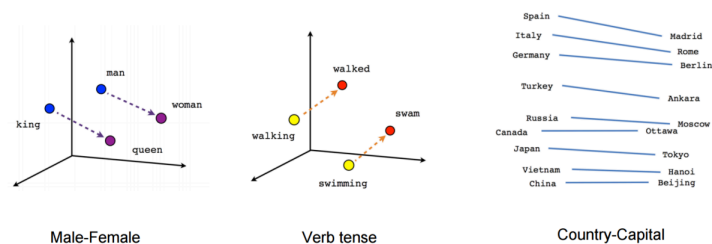
This is useful for meaning-related tasks. We want to have a sense of similarity, relatedness, and semantic role.

In this section we summarize some of these desiderata, drawing on results in the linguistic study of word meaning, which is called lexical semantics.

Vector semantics is the standard way to represent word meaning in NLP, helping vector semantics us model many of the aspects of word meaning we saw in the previous section.

From Counting to Predicting

Instead of counting co-occurrences and then compressing them, Word2Vec trains a model to **predict** context relationships directly. The learned weights become the word embeddings.



There's a logical relationship in this embedding vector space. The vector that represents the difference between woman and man is sort of similar to the one of the difference between queen and king.

Word2Vec: CBOW and Skip-Gram

Word2vec is fast, efficient to train, and the embeddings are shared (Gensim library).

It is static (differently from transformers).

Unlike many NLP tasks such as sentiment analysis or part-of-speech tagging, which rely on expensive, human-annotated **gold standard labels**, Word2Vec utilizes a **self-supervised learning** approach. The actual words in the surrounding text serve as the gold standard for the

prediction. So in the sentence ‘hi how are you doing?’, the gold standard for ‘doing’ is ‘you’. Pretty easy.

Crucially, the ultimate goal isn't to be a good predictor; the model is simply a means to an end. Of course the model will try to predict the correct word (you), but we don't really care about the classification itself, but we care about the weights, the embeddings. After training, the actual **prediction task is discarded**, and only the resulting dense **embedding encoding** (the learned weights or representations) is kept and used for downstream tasks. This self-supervised representation learning ability is what makes the model so powerful and scalable, allowing it to train on massive amounts of unlabelled data, such as the entire web.

Two algorithms are possible:

- **CBOW**: Predict a target word given its surrounding context.
- **Skip-Gram**: Predict each context word from a target word (the version emphasized in class).

Skip-Gram with Negative Sampling (SGNS)

- Positive samples: real context pairs from a sliding window.
- Negative samples: randomly sampled non-neighbors used as “fake” contexts.
- The model increases similarity (dot product) for true pairs and decreases it for noisy ones.

This is essentially logistic regression over dot products between “target” and “context” embeddings.

The Mechanism

1. **Task**: The model is given a word (w , the target) and is tasked with predicting the context words c around it (Skip-Gram).
2. **Gold Standard**: The actual surrounding words in the training text serve as the "gold standard" or true labels.
3. **Training**: The model, which uses simple binary classification with **logistic regression**, learns a set of weights (which are actually the embeddings!) that **maximize the probability of predicting true context words** and **minimize the probability of predicting randomly sampled "negative" words**.
4. **Output**: We don't care about the prediction task itself. We only care about the **word embedding encoding** (W matrix). These learned weights become the dense, distributed vectors that automatically capture the desired semantic relationships because they were optimized to correctly predict context.

... lemon, a tablespoon of apricot jam, a pinch ...

[c1 c2 w c3 c4]			
w	c_{pos}	w	c_{neg}^*
apricot	tablespoon	apricot	aardvark
apricot	of	apricot	my
apricot	jam	apricot	where
apricot	a	apricot	coaxial

$P(+|w, c)$
 $P(-|w, c) = 1 - P(+|w, c)$
Similarity(w, c) $\approx \mathbf{c} \cdot \mathbf{w}$

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(\mathbf{c}_i \cdot \mathbf{w})$$

Maximize dot product between c_{pos} , minimize c_{neg} .

*Negative examples are sampled using weighted $\alpha = 0.75$ unigram count, similar to PPMI.

In this example, the context window is of size 2, in fact you can see that we consider *tablespoon, of, jam, a*.

$$\begin{aligned}
 \text{Similarity}(w, c) &\approx \mathbf{c} \cdot \mathbf{w} \\
 \sigma(\mathbf{c}_i \cdot \mathbf{w}) &= \frac{1}{1 + \exp(-\mathbf{c}_i \cdot \mathbf{w})} \\
 P(+|w, c_{1:L}) &= \prod_{i=1}^L \sigma(\mathbf{c}_i \cdot \mathbf{w}) \\
 \log P(+|w, c_{1:L}) &= \sum_{i=1}^L \log \sigma(\mathbf{c}_i \cdot \mathbf{w})
 \end{aligned}$$

Skip-gram with negative sampling is literally:

A logistic regression classifier that tries to predict

“Is this (w, c) pair a *real* pair from the corpus or a *fake* one?”

It actually does this on a window, so this probability will be given by the product of the individual probabilities (independence assumption).

How is the probability computed? By just applying the usual sigmoid (to squash between 0 and 1) on a score, which will be given by the similarity between the embeddings of the context words and the target word.

Book: In summary, skip-gram trains a probabilistic classifier that, given a test target word w and its context window of L words $c_1:L$, assigns a probability based on how similar this context window is to the target word. The probability is based on applying the logistic (sigmoid) function to the dot product of the embeddings of the target word with each context word. To compute this probability, we just need embeddings for each target word and context word in the vocabulary.

Word2Vec learns **two sets of embeddings**:

Symbol	Meaning	Acts Like in Logistic Regression
w	target word embedding	feature vector (x)
c	context word embedding	weight vector (θ)

So the logistic regression score becomes:

$$\text{score}(w, c) = \text{similarity}(w, c) = w \cdot c = \sum_i w_i c_i$$

Logistic regression: $\theta^T x$. Here, in Skip-gram, it become $c^T w$.

$P(+ | w, c) = \sigma(c \cdot w)$ is the probability that c is truly a context of w .

$P(- | w, c) = 1 - \sigma(c \cdot w)$

Word2Vec has **two embedding matrices**:

- W — word embeddings (target words)
- C — context embeddings (context words)

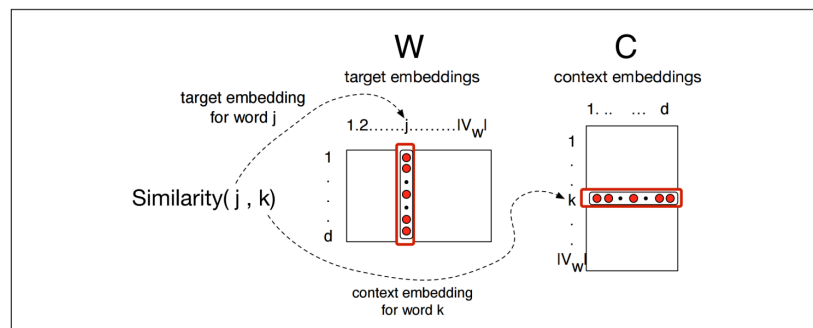
During training:

- For **positive sample** (w, c_{pos}) → You maximise the dot product $w \cdot c_{pos}$ (and therefore maximise $\sigma(w \cdot c)$)
- For **negative sample** (w, c_{neg}) → You minimise the dot product $w \cdot c_{neg}$

Thus, *all* words have:

- a word vector (in W)
- a context vector (in C)

Both matrices are updated.



- W has as d rows, where d is the dimensionality of the embedding vectors.
 W has $|W|$ columns, one for each word in the vocabulary.
 W contains the representations of the words when a word is treated as the central item being focused on, i.e. the target.

- C has as $|W|$ rows and d columns. This vector is used as a **weight matrix** (the classifier weights) to predict how likely word k is to be in the context of any other word j .

During the Skip-Gram training, the model learns two matrices: Target Embeddings (W) and Context Embeddings (C). When you stop training and want to use the embeddings, you must choose how to combine them:

- **Option A: Use W only**, i.e. discard C entirely and use the vector w_i from the W matrix as the final embedding for word i . Usually this is what we do.
- **Option B: Combine W and C** , i.e. sum or concatenate the vectors $w_i + c_i$

Properties of Embeddings

Embeddings often encode **semantic relations linearly**.

Examples:

- **Analogies:** king – man + woman $\approx$ queen
- **Regularities:** verb tenses, capital–country pairs, pluralizations

High-dimensional embeddings can be projected into 2D (e.g., via t-SNE) to reveal clusters such as fruits, animals, or verbs.

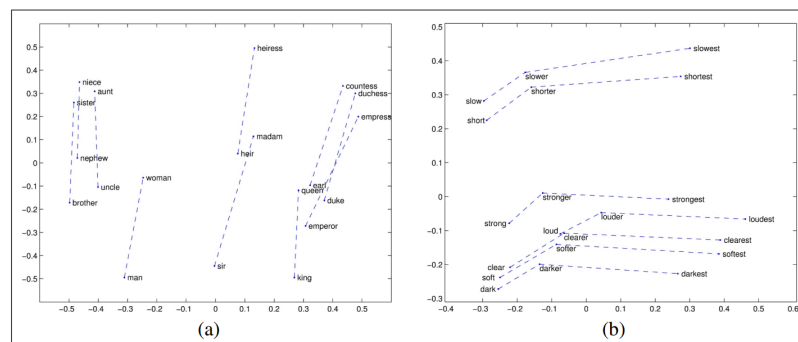


Figure 6.16 Relational properties of the GloVe vector space, shown by projecting vectors onto two dimensions. (a) king – man + woman is close to queen. (b) offsets seem to capture comparative and superlative morphology (Pennington et al., 2014).

Bias

Because embeddings are trained on human-generated text, they absorb and sometimes amplify human biases (e.g., gender stereotypes).

Evaluation

Evaluation takes two forms:

- **Extrinsic:** The most crucial approach. It involves using the vectors in a **downstream Natural Language Processing (NLP) task** and observing whether this **improves performance** compared to another baseline or model.
- **Intrinsic:** Immediate, task-specific evaluations that test the quality of vectors on well-defined tasks, often related to semantic meaning or relationships.

Analogy task (intrinsic)

The goal is to find a vector x (which is a known word embedding) that is the closest match to the resulting vector produced by the subtraction and addition operation.

$$\hat{b} = \arg \min_x \text{distance}(x, \mathbf{a} - \mathbf{a} + \mathbf{b})$$

Example: King - Man + Woman = Queen

The equation tests the linearity of the **Male-Female** axis and the **Royalty** concept.

- **Isolate the Relationship:** The model calculates the vector difference between king and man. The resulting vector captures the **gender-neutral abstract concept of "Royalty"**.
- **Transfer the Relationship:** This difference vector is then added to the woman vector.
- **Find the Nearest Neighbor:** The model searches the entire vocabulary of vectors (using the *arg min* to find the known word vector x that has the minimum distance (e.g., using cosine distance) to this resulting target vector.

- **Successful Outcome:** If the embedding space is well-trained, the vector closest to the result should be queen.

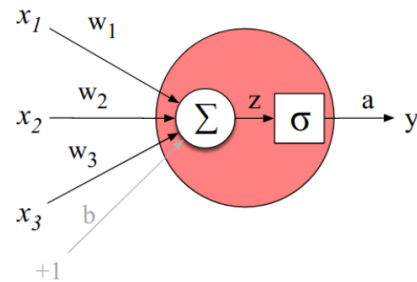
This same arithmetic principle is used to test other relationships, such as **Verb Tenses** (**walked** – **walking** + **swimming** $\approx$ **swam**) or **Country-Capital** relations.

Neural Models of Language

The Deep Learning Shift

Deep Learning replaces hand-crafted linguistic features with **representation learning**. Neural networks discover abstractions automatically from raw inputs.

A neuron computes a weighted sum of inputs, adds a bias, and passes it through a non-linear activation function such as sigmoid, tanh, or ReLU.



Feedforward Neural Networks (FFNN)

To model complex, non-linear relationships (like XOR), we stack layers of neurons.

Without non-linearities, multiple layers collapse mathematically into one, so activations are essential.

Neural networks can be used for:

- **Classification:** embeddings $\rightarrow$ hidden layer $\rightarrow$ softmax

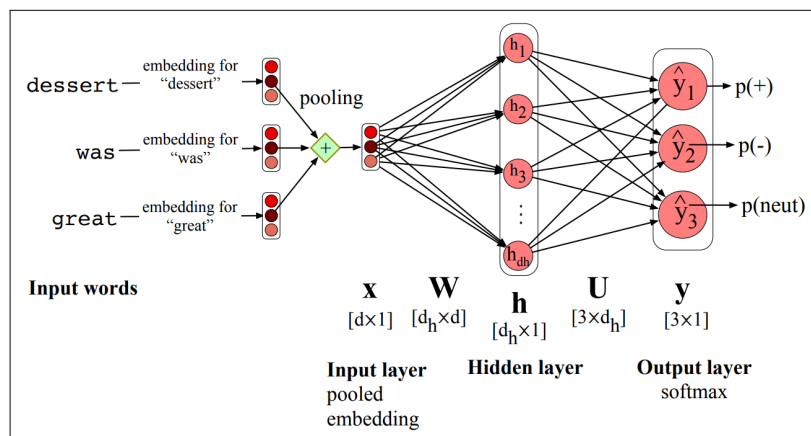
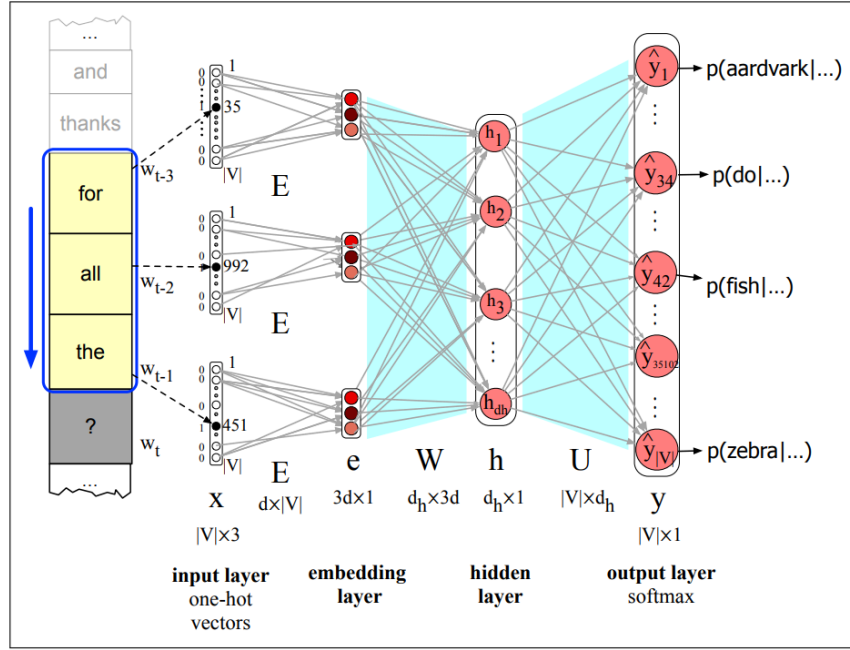


Figure 7.11 Feedforward network sentiment analysis using a pooled embedding of the input words.

- **Language modeling:** predict the next word given N previous ones, using embeddings of the preceding window and a softmax over the whole vocabulary

$$P(w_t | w_{t-N+1}, \dots, w_{t-1})$$



This example shows how a Neural Probabilistic Language Model (NNLM) predicts the next word ("the") given a window of $N = 3$ previous words ("thanks for all").

1. Setup

Parameter	Value
Context ($w_{t-3}, w_{t-2}, w_{t-1}$)	"thanks", "for", "all"
Vocabulary Size V	V
Embedding Dim d	2
Hidden Size d_h	3
One-Hot Inputs $\mathbf{x}$	Sparse vectors (e.g., $\mathbf{x}_{\text{thanks}} = [1, 0, 0, 0, 0]^T$)

2. Embedding Lookup and Concatenation

The one-hot vectors are multiplied by the Embedding Matrix $\mathbf{E}$ (2×5) to get dense vectors, which are then concatenated.

$$\mathbf{e} = [\mathbf{E}\mathbf{x}_{t-3}; \mathbf{E}\mathbf{x}_{t-2}; \mathbf{E}\mathbf{x}_{t-1}]$$

$$\mathbf{e} = \begin{bmatrix} 0.5 \\ 0.2 \\ 0.1 \\ 0.8 \\ 0.9 \\ -0.4 \end{bmatrix}$$

3. Hidden Layer Calculation

The concatenated vector $\mathbf{e}$ is processed through the hidden layer weights $\mathbf{W}$ (3×6) and activation function σ (tanh).

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{e} + \mathbf{b})$$

- **Linear Output ($\mathbf{W}\mathbf{e} + \mathbf{b}$):** $[0.5, 1.0, 1.0]^T$
- **Activation (tanh):** $\mathbf{h} = [0.46, 0.76, 0.76]^T$

4. Output Projection and Softmax

The hidden state $\mathbf{h}$ is projected to the vocabulary size using the output matrix $\mathbf{U}$ (5×3) to get the logits $\mathbf{z}$

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\mathbf{z} = [0.046, 0.122, 0.38, \mathbf{1.52}, 0.198]^T$$

Finally, the **Softmax** function is applied to convert the logits $\mathbf{z}$ into a probability distribution $\hat{\mathbf{y}}$ that sums to 1.

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

These models bring smooth probability estimates and can generalize better than n-gram models.

Practical Applications: Evaluation, Debugging, and Robustness (Expanded)

Why High Accuracy Isn’t Enough

Models often learn shortcuts instead of the intended concepts.

Example: A toxic-content classifier may wrongly treat identity terms (“gay”, “Muslim”) as signals of toxicity because of biased training data. This leads to false positives on neutral statements.

Explainability Tools

To understand *why* a model behaved a certain way, we use:

- **LIME:** Perturbs the input (e.g., removes individual words) to reveal which pieces influence predictions.
- **SHAP:** Assigns importance values to each feature, showing how each word contributes toward or away from a particular class.

These tools help diagnose reliance on unintended features, dataset bias, or unstable predictions.

Contrast Sets and Adversarial Testing

Standard test sets resemble the training set too closely.

Contrast sets involve minimally modifying examples—changing a sentiment word, adding negation, swapping entities—to test whether the model truly understands the underlying concept.

Adversarial examples deliberately probe model blind spots, such as changing writing style to evade authorship detection.

Data Augmentation for Robustness

Easy Data Augmentation (EDA) includes:

- Synonym replacement
- Random insertion
- Random swap
- Random deletion

These methods help models generalize, especially on small datasets.

However, synonym replacement must be used carefully—context determines whether substitutions are meaningful or harmful (e.g., scientific terms, domain-specific vocabulary).

This chapter marks the big conceptual transition from **global document representations** (Bag-of-Words, embeddings, tf-idf) to **token-level, structure-aware models**. After weeks of treating text as unordered collections, we now acknowledge a crucial fact:

Language is sequential. Word order matters. Meaning unfolds over time.

This allows us to tackle **word-level prediction tasks** such as POS tagging, Named Entity Recognition (NER), relation extraction, and event extraction — the foundations of information extraction.

Up until now:

- We predicted **one label per document**.
- Even embeddings were **global** representations.
- The models ignored **the sentence as a structure**.

But many real NLP tasks require knowing **what each word does**, not just what the whole text is about.

Examples:

- “*back*” can be an adjective (“back seat”), noun, verb, particle, or adverb depending on context.
- A named entity like “**United**” can be an **organization**, a **location**, or part of an airline name.
- Relations depend on sequence order:
“**United**, a unit of **UAL Corp**” expresses the relation *part-of*.

Therefore, we shift from **global classification** to **sequence classification**, where the input is:

Sequence in: $x_1, x_2, x_3, \dots, x_n$

Sequence out: $y_1, y_2, y_3, \dots, y_n$

Information Extraction

- **The Goal (Information Extraction):** We want to turn unstructured text into structured databases (e.g., extracting dates, names, and relationships to build a knowledge graph).
- **The Method (Sequence Labeling):** To achieve that goal, we cannot treat a document as a "bag of words" anymore. We must analyze the **sequence** to understand that "United" followed by "Airlines" is an Organization, but "United" followed by "States" is a Location.

The logical flow of this chapter is:

- **Phase I: The Problem (Ambiguity)** We used to classify whole documents (e.g., "Is this email spam?"). But real tasks require knowing what individual words mean in context.
 - **The Challenge:** Words are ambiguous. "Back" can be a noun, verb, adjective, or particle depending entirely on the **sequence** of words around it.
 - **The Solution:** We need a model that reads $x_1, x_2, \dots$ and outputs tags $y_1, y_2, \dots$.
- **Phase II: The Foundations (NER & POS)** Before we can extract complex facts (like "Apple acquired Beats"), we must automate two basic tasks:
 - **POS Tagging:** Understanding grammar (Noun, Verb, Adjective). This helps disambiguate meaning.
 - **NER (Named Entity Recognition):** Identifying the "actors" (People, Organizations, Locations).
- **Phase III: The Mechanics (HMMs & CRFs)** How do we actually build a system to do POS tagging and NER?
 - **HMMs (Hidden Markov Models):** The probabilistic approach. We assume hidden tags cause the observed words. It uses the Viterbi algorithm to find the most likely path of tags.
 - **CRFs (Conditional Random Fields):** The modern, discriminative approach. HMMs are too restrictive (can't look at "neighboring words" or "capitalization"). CRFs allow us to use **arbitrary features** (like "does this word end in -ing?") to make better sequence predictions.
- **Phase IV: The Payoff (Information Extraction)** Once we have the sequences tagged (thanks to HMMs/CRFs), we can finally perform **Information Extraction**, which allows us to:
 - **Extract Relations:** "United is a unit of UAL Corp" → `(United, Part-Of, UAL Corp)`.
 - **Extract Events:** Detecting timestamps and participants.

Where Can Information Live?

Information can exist at **multiple levels**, all of which sequential models may help with.

- **Meta-data Level.** Context surrounding the text:
 - Time when it was written
 - Who sent it / who received it
 - Upvotes / likes (normative context)
 - Geo-locationThese enrich interpretation — a tweet at 2am vs 2pm conveys different things.
- **Collection Level.** From patterns across the collection:
 - Topics (LDA, Naive Bayes topic distribution)
 - Dialogues → conversation structure
 - Semantics → from embeddings
 - Paraphrases → summaries or abstracts

Different corpora = different semantic universes (e.g., Reddit vs Wikipedia embeddings).

- **Document Level.** Labels for entire documents:
 - Sentiment (positive/negative)
 - Emotions (anger, joy, sadness)
 - Sarcasm

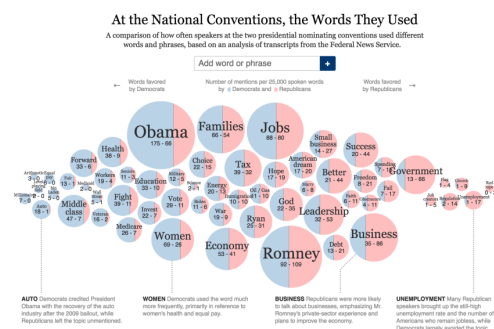
- These are classic classification tasks.

- Syntactic parsing
- Disambiguating word senses
- Detecting entities
- Predicting timestamps from text

These require **token-level** reasoning → **sequence models**.

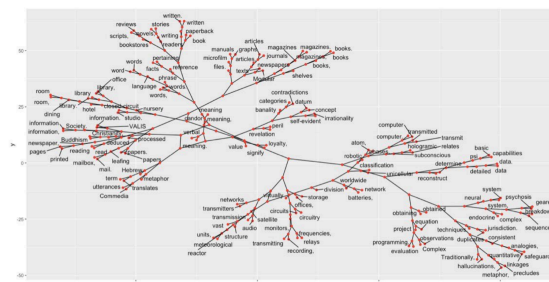
New York Times political speeches tool

Interactive interface showing which political party used which words during conventions.



Visualizes topic-term relations as probability “blobs” — shows how topics emerge in text.

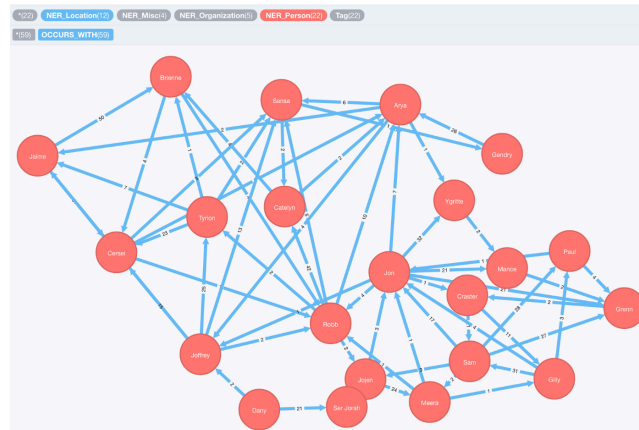
Hierarchical clusters of embedding space distances — revealing semantic ontologies.



Using sentiment of Reddit posts (WallStreetBets) to understand hype cycles or pump-and-dump schemes.

- Nodes = characters
- Edges = co-occurrence in the same passage
- Shows relations discovered from texts.

This is **information extraction** in practice.

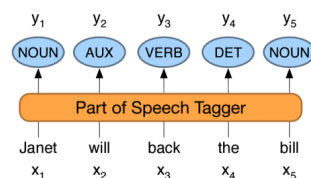


Sequence Classification: The General Setup

Sequence tasks assign **one label per token**.



Given a sequence $S = x_1, x_2, \dots, x_n$, predict a label **for each position**, i.e. $\hat{y} = y_1, y_2, \dots, y_n$. Example, POS Tagging:



Example: POS tagging sentence

“Janet will back the bill.”

Token	POS Tag
Janet	NNP (Proper Noun)
will	MD (Modal)
back	VB / NN / RP / ... (ambiguous!)
the	DT
bill	NN

The model must consider:

- Local context
- Word identity
- Surrounding tags

This cannot be done with bag-of-words or logistic regression alone.

Part-of-Speech (POS) Tagging: A Motivating Case

Human accuracy: ~97%

Models (even simple ones) achieve similar accuracy → but only because most words are *easy*.

- **85%** of word **types** are unambiguous
- But **60%** of actual **tokens** in running text are ambiguous
- Majority baseline = **92%** (!)

Meaning:

- The real challenge is the ~5–8% ambiguous cases

- Models must handle *contextual disambiguation*

Example: The word “*back*”

It can be:

Sentence	Meaning	POS
“earnings growth took a back seat”	adjective	JJ
“in the back of the house”	noun	NN
“senators back the bill”	verb (present)	VBP
“began to back toward the door”	verb (base)	VB
“buy back debt”	particle	RP
“back then”	adverb	RB

A model must consider:

- previous words
- next words
- likelihood of tag sequences

→ **Sequence models handle this naturally.**

Named Entity Recognition (NER)

Named entity recognition (NER) is a component of natural language processing (NLP) that *identifies predefined categories of objects in a body of text*. **Named Entities** are **real-world objects like people, organizations, locations, dates, and monetary values that can be identified and categorized in text**, a process called Named Entity Recognition (NER). NER converts unstructured text into structured information, helping systems understand key details for applications like search engines, chatbots, and summarization tools by tagging words into predefined types (e.g., PERSON, ORG, GPE).

Goal:

- Detect spans referring to people, places, organizations, etc.

Example:

“[ORG United], a unit of [ORG UAL Corp.], said fares increased by [MONEY \$6] effective [TIME Thursday] on routes from [LOC Chicago] to [LOC Dallas] ...”

Why sequential models? Because NER labels depend on:

- previous labels
- long entities spanning multiple words
- boundaries (“begin” vs “inside”)

Sequence Models: Markov Models & Hidden Markov Models

A Markov model tells us the probability of sequences of random variables. The variables are called states.

A simple Markov model assumes:

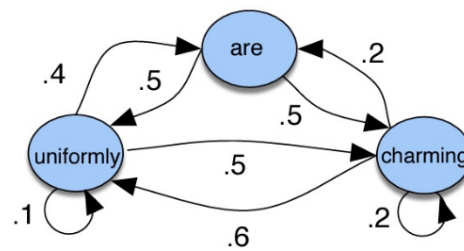
next state depends only on the current state

Generally, this assumption enables reasoning and computation with the model that would otherwise be intractable.

This is near-identical to bigram language modeling:

- $P(q_i = a | q_1, \dots, q_{i-1}) = P(q_i = a | q_{i-1})$

Edges are $P(w_i | w_j)$.



We start by analyzing a corpus of text to determine the probabilities of transitioning from one word to another. For example, suppose we have the following sentence:

The quick brown fox jumps over the lazy dog.

We can create a Markov chain by treating each word as a state and analyzing the probability of transitioning from one word to another. For example, we might find that the probability of transitioning from "the" to "quick" is 0.5, the probability of transitioning from "quick" to "brown" is 1.0, and so on based on large corpus text data study. Once we have computed the transition probabilities, we can generate a new sentence by starting with an initial word and randomly selecting the next word based on the transition probabilities.

Transition matrix: The fundamental mathematical concept of a Markov chain is the transition matrix. This is a square matrix that describes the probability of moving from one state to another. If there are n states in the Markov chain, the transition matrix will be an $n \times n$ matrix, where each element (i, j) of the matrix represents the probability of moving from state i to state j . The sum of each row of the transition matrix must be 1, as the probabilities of moving to each state from the current state must add up to 1.

The problem is that a normal Markov model only represents a sequence of observable states. But in NLP, the states we care about are *not* observable — they are the *tags*. This background representation doesn't really hold up... we're not only looking at the sequence, we also have tags associated with the sequence.

A normal Markov model cannot represent:

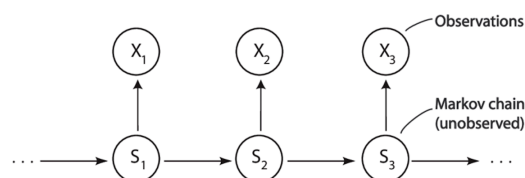
- that a verb tends to follow a modal
- that an adjective tends to precede a noun
- etc.

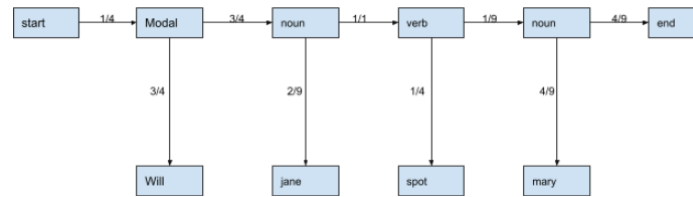
Because a normal Markov model only models transitions between **observable things** (words).

But transitions between words are nearly useless:

- the vocabulary is huge
- most word–word transitions never occur
- the transition matrix would be thousands $\times$ thousands
- and it wouldn't capture the grammar anyway

Hidden Markov Models (HMMs)





What the HMM assumes is that the data that you observed which are called the observed states are the items you're observing in time (whether those are characters or words etc), and these observed states arise from a set of hidden states which you do not observe but which you are interested in tagging or labeling.

HMMs jointly model:

- **Hidden tags** (POS, NER labels)
- **Observed words**

We assume that one causes the other, meaning that the **state sequence** (tags) generates the **observation sequence** (words)

This is what allows the model to say:

- if the tag is **NN**, word is likely “bill”, “idea”, “table”...
- if the tag is **VB**, word is likely “eat”, “run”, “back”...
- if the tag is **MD**, word is likely “will”, “can”, “might”...

Without this hidden layer, the model has no access to grammatical structure.

$Q = q_1 q_2 \dots q_N$	a set of N states
$A = a_{11} \dots a_{ij} \dots a_{NN}$	a transition probability matrix A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \dots o_T$	a sequence of T observations , each one drawn from a vocabulary $V = v_1, v_2, \dots, v_V$
$B = b_i(o_t)$	a sequence of observation likelihoods , also called emission probabilities , each expressing the probability of an observation o_t being generated from a state q_i
$\pi = \pi_1, \pi_2, \dots, \pi_N$	an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

i States are **hidden events** (labels), and ‘observing’ refers to **observed events** (words).

HMM parameters:

1. **Transition probabilities** A : $P(tag_i | tag_{i-1}) = \frac{C(tag_{i-1}, tag_i)}{C(tag_{i-1})}$
These model grammatical patterns, e.g.: DT $\rightarrow$ NN, MD $\rightarrow$ VB, JJ $\rightarrow$ NN
This is the **Markov** part of the Hidden Markov Model.
2. **Observation likelihoods (also called emission probabilities)** B : $P(word_i | tag_i) = \frac{C(tag_i, word_i)}{C(tag_i)}$
 B gives additional information if a particular observation is generated from a particular state.
Emissions tie words to tags.
3. **Initial probabilities** π over the states. π_i is the probability that the Markov chain will start in state i .
 $\sum_{i=1}^n \pi_i = 1$

Decoding

Decoding refers to predicting a particular tag given our model with the two probability matrices A and B .

Just like Naive Bayes:

- Count tag–tag transitions
- Count tag–word co-occurrences
- Convert to probabilities

Formally: given as input an HMM $\lambda = (A, B)$ and a sequence of observations $O = o_1, o_2, \dots, o_T$, find the most probable sequence of states $Q = q_1 q_2 q_3 \dots q_T$. This give (t stands for the tag and w stands for the word):

$$\hat{t}_{1:n} = \arg \max_{t_1 \dots t_n} P(t_1 \dots t_n | w_1 \dots w_n) \approx [Bayes] \approx \arg \max_{t_1 \dots t_n} \prod_{i=1}^n \overbrace{P(w_i | t_i)}^{\text{emission}} \overbrace{P(t_i | t_{i-1})}^{\text{transition}}$$

i.e. a product of emission times transition.

Straightforward maximum-likelihood estimation.

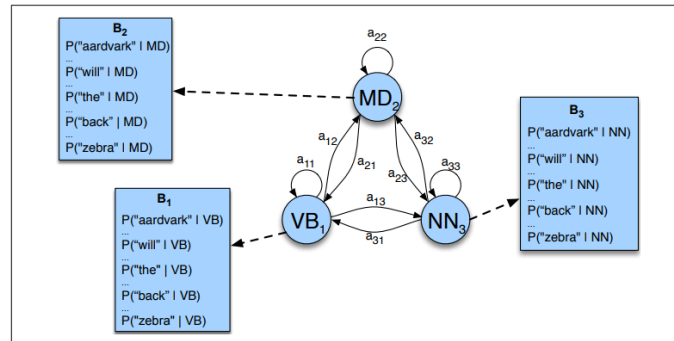


Figure 17.9 An illustration of the two parts of an HMM representation: the A transition probabilities used to compute the prior probability, and the B observation likelihoods that are associated with each state, one likelihood for each possible observation word.

Decoding: The Viterbi Algorithm

Given a sentence:

- compute the most probable tag sequence

The professor compared it to **Minimum Edit Distance**. We generated a matrix of operations (where there are multiple operations we could do, but only one ideal one).

- forward pass fills probabilities
- backward pass reconstructs best sequence

Input

- Observation space $O = \{o_1, o_2, \dots, o_n\}$.
- State space $S = \{s_1, s_2, \dots, s_K\}$.
- Initial probabilities $\Pi = (\pi_1, \pi_2, \dots, \pi_K)$ such that π_i stores the probability that $x_1 = s_i$.
- Observations $Y = (y_1, y_2, \dots, y_t)$ such that $y_t = o_i$ if observation at time t is o_i .
- Transitions $A (K \times K)$ such that $A_{i,j}$ stores the transition probabilities from state s_i to s_j .
- Emissions $B (K \times N)$ such that $B_{i,j}$ stores the probability of observing o_j from state s_i .



States are **hidden events** (labels), and 'observing' refers to **observed events** (words).

Output

- The most likely hidden state sequence $X = (x_1, x_2, \dots, x_T)$.



T_n here denotes the trellises we are filling (one for probabilities, one for backpointers), and k the prior state. Note that because we fill T_1 in the first loop with π (for NLP, usually this is $P(\text{something given } \langle s \rangle)$) there is always a prior state. In the second loop block it's implicitly set to the result of $\arg \max$.

See the next pseudocode for the vectorized version.

```

function VITERBI( $O, S, \Pi, Y, A, B$ ) :  $X$ 
  for each state  $i = 1, 2, \dots, K$  do
     $T_1[i, 1] \leftarrow \pi_i \cdot B_{iy_1}$ 
     $T_2[i, 1] \leftarrow 0$ 
  end for
  for each observation  $j = 2, 3, \dots, T$  do
    for each observation  $i = 1, 2, \dots, K$  do
       $T_1[i, j] \leftarrow \max_k (T_1[k, j-1] \cdot A_{ki} \cdot B_{iy_j})$ 
       $T_2[i, j] \leftarrow \arg \max_k (T_1[k, j-1] \cdot A_{ki} \cdot B_{iy_j})$ 
    end for
  end for
  for  $j = T, T-1, \dots, 2$  do
     $z_{j-1} \leftarrow T_2[z_j, j]$ 
     $x_{j-1} \leftarrow s_{z_{j-1}}$ 
  end for
  return  $X$ 
end function

```

Pseudocode:

```

def viterbi(observations, A, B, pi):
    """
    observations: list of integers (indexes into B), e.g. [0,1,2,3,...]
    A: transition matrix, shape (N_states, N_states)
        A[i][j] = P(tag_j | tag_i)
    B: emission matrix, shape (N_states, N_observations)
        B[i][o] = P(observation o | tag_i)
    pi: initial probabilities for each state (vector of length N_states)
    """

    T = len(observations)          # number of words in the sentence
    N = len(pi)                    # number of possible tags / hidden states

    # viterbi[s][t] = best probability of any path that ends in state s at time t
    viterbi = [[0.0 for t in range(T)] for s in range(N)]

    # backpointer[s][t] = argmax previous state that gave the best path into s
    backpointer = [[0 for t in range(T)] for s in range(N)]

    # -----
    # 1. INITIALIZATION (t = 0)
    # -----
    for s in range(N):
        # Best path ending in state s at time 0 is:
        #   initial_prob * emission_prob(word_0 | state_s)
        viterbi[s][0] = pi[s] * B[s][observations[0]]
        backpointer[s][0] = 0 # no previous state at t=0

    # -----
    # 2. VITERBI RECURSION (t > 0)
    # -----
    for t in range(1, T): # for each word after the first
        for s in range(N): # for each possible current tag/state

            # We need to compute:
            # viterbi[s][t] = max over previous states p of:
            #   viterbi[p][t-1] * A[p][s]
            #

```

```

# We store both the maximum value and the best previous state.

best_prev_state = 0
best_prev_value = 0.0

for p in range(N): # loop over all previous states
    candidate = viterbi[p][t-1] * A[p][s]

    # Keep the largest incoming probability
    if candidate > best_prev_value:
        best_prev_value = candidate
        best_prev_state = p

# After finding the best previous state:
# multiply by the emission probability of the current word
viterbi[s][t] = best_prev_value * B[s][observations[t]]

# store the best previous state for backtracking
backpointer[s][t] = best_prev_state

# -----
# 3. TERMINATION – choose final state with max probability
# -----
best_final_state = 0
best_final_prob = 0.0

for s in range(N):
    if viterbi[s][T-1] > best_final_prob:
        best_final_prob = viterbi[s][T-1]
        best_final_state = s

# -----
# 4. BACKTRACE to recover best tag sequence
# -----
best_path = [0] * T
best_path[T-1] = best_final_state

# walk backwards following the backpointers
for t in range(T-2, -1, -1):
    best_path[t] = backpointer[best_path[t+1]][t+1]

return best_path

```

1. Initialization (t = 0)

For the first word, we compute for each possible tag: $v_1(s) = \pi(s) \cdot b_s(o_1)$

This means:

- Probability of starting in tag s
- AND tag s emitting the first word

No transitions yet, because we're at the first word.

2. Recursion (t ≥ 1)

For each time step t and each tag s :

1. Look at **all** tag values p from the previous word → compute: $v_{t-1}(p) \cdot a_{p \rightarrow s}$
2. Pick the **maximum** (the most likely previous tag)
3. Multiply that max by the emission probability of the current word:

$$v_t(s) = \max_p (v_{t-1}(p) a_{p \rightarrow s}) \cdot b_s(o_t)$$

4. Store the **argmax** in the backpointer.

This enforces that we keep not just the best score but also remember **which previous tag created that best path**, so we can reconstruct the whole path later.

3. Termination

At the last time step T :

- Pick the state whose Viterbi value $v_T(s)$ is the largest.
- That state is the final tag in the optimal sequence.

4. Backtrace

Walk backwards through **backpointer**:

$$\text{state}_t = \text{backpointer}[\text{state}_{t+1}][t + 1]$$

Until we reach the first word.

This reconstructs the highest-probability path.

Let's tag the sentence *Janet will back the bill*; the goal is the correct series of tags (see also Fig. 17.11):

(17.20) Janet/NNP will/MD back/VB the/DT bill/NN

	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 17.12 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968. <s> is the start token.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 17.13 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

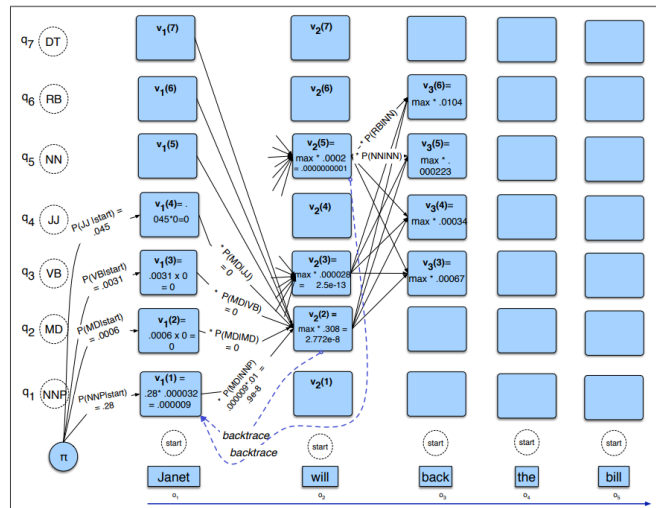


Figure 17.14 The first few entries in the individual state columns for the Viterbi algorithm. Each cell keeps the probability of the best path so far and a pointer to the previous cell along that path. We have only filled out columns 1 and 2: to avoid clutter most cells with value 0 are left empty. The rest is left as an exercise for the reader. After the cells are filled in, backtracing from the *end* state, we should be able to reconstruct the correct state sequence NNP MD VB DT NN.

We start with some prior probabilities, encoded in π . In this example, π is basically given by the first row of the A matrix, since $\langle s \rangle$ represents the start.

Then, for example, the probability that the first word (Janet) is a NNP is given by the product between $P(\text{Janet}|\text{NNP})$ and $P(\text{NNP}|\langle s \rangle)$.

The probability that Janet is VB is 0 because in the corpus, i.e. our training data, we never observed Janet being a VB. Therefore the co-occurrence of Janet and VB is 0, and so is the probability $P(\text{Janet}|\text{VB})$.

Now for the next words, in general, at time t , for each possible tag j (NNP, MD, VB, ...), you compute:

$$v_t(j) = \max_i (v_{t-1}(i) \cdot a_{i \rightarrow j}) \cdot b_j(o_t)$$

This breaks into two parts: first, for every **previous tag** i : $\text{incoming}(i \rightarrow j) = v_{t-1}(i) \cdot a_{i \rightarrow j}$

- Take the **score of the best path that ended in tag i at time $t-1$**
- Multiply by the **transition probability** from $i \rightarrow j$

You do this for **all possible i** (all tags).

The second part is choosing **the maximum of those incoming probabilities**: $\max_i (v_{t-1}(i) \cdot a_{i \rightarrow j})$. This is the “Viterbi choice.”

It says:

“Which previous tag gives me the highest probability of ending in tag j now?”

This max tells you:

- **the best previous tag** (store it in the backpointer)
- **the score of that best path up to tag j , before considering the emission**

Once you know the best incoming path, you multiply by: $b(j, o_t)$

This is the probability of the current word being produced by tag j .

Exercise

Given A and B , calculate the forwarding step for Viterbi, i.e. the late lattice, for the following sequence: `you do like apple`. You only have to consider the max of the previous column. (If states are zero, you can just skip them in any other calculations, as the results will always be zero.) When a column is all zeroes, you can stop.

▼ Solution (Full)

Given:

label →	<s>	ADP	AUX	NOUN	PRON	VERB	</s>
↓ previous label							

<s>	0	0	0.333	0.333	0.333	0	0
ADP	0	0	0	1.000	0	0	0
AUX	0	0	0	0	1.000	0	0
NOUN	0	0.167	0	0.333	0	0	0.500
PRON	0	0	0	0	0	1.000	0
VERB	0	1.000	0	0	0	0	0
</s>	0	0	0	0	0	0	0

label word	<s>	ADP	AUX	NOUN	PRON	VERB	</s>
<s>	0	0	0	0	0	0	0
apple	0	0	0	0.500	0	0	0
do	0	0	1.000	0	0	0	0
i	0	0	0	0	0.500	0	0
like	0	1.000	0	0	0	1.000	0
pie	0	0	0	0.5000	0	0	0
you	0	0	0	0	0.500	0	0
</s>	0	0	0	0	0	0	0

We get:

input S predictions X	you	do	like	apple
ADP	$P(\text{ADP} \text{<s>}) * P(\text{you} \text{ADP}) = 0.000 * 0.000 = 0.000$	$P(\text{ADP} \text{max t-1 = PRON}) * P(\text{do} \text{ADP}) = 0.000 * 0.000 = 0.000$		
AUX	$P(\text{AUX} \text{<s>}) * P(\text{you} \text{AUX}) = 0.333 * 0.000 = 0.000$	$P(\text{AUX} \text{max t-1 = PRON}) * P(\text{do} \text{AUX}) = 0.000 * 1.000 = 0.000$		
NOUN	$P(\text{NOUN} \text{<s>}) * P(\text{you} \text{NOUN}) = 0.333 * 0.000 = 0.000$	$P(\text{NOUN} \text{max t-1 = PRON}) * P(\text{do} \text{NOUN}) = 0.000 * 0.000 = 0.000$		
PRON	$P(\text{PRON} \text{<s>}) * P(\text{you} \text{PRON}) = 0.333 * 0.500 = 0.167$	$P(\text{PRON} \text{max t-1 = PRON}) * P(\text{do} \text{PRON}) = 0.000 * 0.000 = 0.000$		
VERB	$P(\text{VERB} \text{<s>}) * P(\text{you} \text{VERB}) = 0.000 * 0.000 = 0.000$	$P(\text{VERB} \text{max t-1 = PRON}) * P(\text{do} \text{VERB}) = 0.000 * 0.000 = 0.000$		



If we had more observations, or applied smoothing (😊) some of these probabilities in the second column would not have been zero.

Conditional Random Fields (CRFs)

Given a sequence of words X and a sequence of tags Y , we want the most likely sequence of tags $\hat{Y} = \arg \max_Y P(Y|X)$.

Both HMMs and CRFs try to solve this equation, but they compute $P(Y|X)$ in very different ways.

An HMM does not model it directly, but instead relies on Bayes rule to decompose it into $\hat{Y} = \arg \max_Y P(X|Y)P(Y)$.

HMMs must obey generative assumptions:

- Words depend **only** on their tag (no access to neighboring words)
- Tags depend **only** on previous tag (first-order Markov)
- Cannot add arbitrary features (like “suffix = -ing”, “word is capitalized”, “previous word is ‘the’”, etc.)

These are big restrictions.

In general it's hard for generative models like HMMs to add arbitrary features directly into the model in a clean way. We've already seen a model for combining arbitrary features in a principled way: log-linear models like the logistic regression model. But logistic regression isn't a sequence model; it assigns a class to a single observation.

CRFs fix HMM weaknesses by modeling $P(Y|X)$ directly (similar to **logistic regression** but for sequences). No Bayes rule, no generative model, no need to model $P(X)$. This allows CRFs to use **arbitrary features of the entire X**.

A CRF is a log-linear model that assigns a probability to an entire output (tag) sequence Y , out of all possible sequences Y , given the entire input (word) sequence X .

A big difference between HMMs and CRFs is that the former are generative models, while CRFs are merely discriminative, focusing only on predicting the label sequence Y given the observed sequence X . They **don't care about modeling how X is generated**; they just care about making the best prediction of Y .

Note: we are only going to deal with linear CRFs, i.e. the y elements have to be close to each other, otherwise there are too many possible sequences and it gets too complicated and expensive. But still, all the possible connections between any x and any y is valid.

We want to figure out the **probability of a hidden state** (like a part of speech) given the observed data. In other words:

Given that we have observed some data X , what is the relationship between that data and a specific hidden state Y ?

The answer in CRFs comes from **feature functions**, which are the biggest strengths of these CRFs. They allow us to encode **anything we think might help the model learn sequential dependencies**. Feature functions can be **any function** as long as their inputs are limited to four things: y_{i-1}, y_i, X, i .

- For K features, we use some function F_k to map it into a **global feature** vector to assign weight w_k .

$$p(Y | X) = \frac{\exp\left(\sum_{k=1}^K w_k F_k(X, Y)\right)}{\sum_{Y' \in \mathcal{Y}} \exp\left(\sum_{k=1}^K w_k F_k(X, Y')\right)}$$

- Global features $\rightarrow$ **local features**; use the previous output, the current one, the whole input, and the index:

$$F_k(X, Y) = \sum_{i=1}^n f_k(y_{i-1}, y_i, X, i)$$

Key advantages:

- Can use **arbitrary features** (prefixes, suffixes, capitalization, embeddings, gazetteers...)
- Use **the entire input** and previous labels
- No generative assumptions
- Much better at handling unknown words or rare patterns

Each feature (local feature type) has a learned weight w_k .

The linear-chain CRF defines a score: $\text{score}(Y, X) = \sum_k w_k F_k(Y, X)$

Since F_k is a sum of f_k over the whole sequence:

$$F_k(Y, X) = \sum_i f_k(y_{i-1}, y_i, X, i)$$

We can rewrite the score more transparently:

$$\text{score}(Y, X) = \sum_{i=1}^n \sum_k w_k f_k(y_{i-1}, y_i, X, i)$$

This expresses:

- At each position i
- Many feature functions f_k fire (or not)
- Each multiplied by its weight w_k
- Summed across the sequence

Again, in a linear-chain CRF, each local feature f_k at position i can depend on any information from: (y_{i-1}, y_i, X, i) . So some legal features representing common situations might be the following:

- $1\{x_i = \textit{the}, y_i = \textit{DET}\}$
- $1\{y_i = \textit{PROPN}, x_{i+1} = \textit{Street}, y_{i-1} = \textit{NUM}\}$
- $1\{y_i = \textit{VERB}, y_{i-1} = \textit{AUX}\}$

For simplicity, we'll assume all CRF features take on the value 1 or 0.

Feature Templates

- Word shape ("Xxxxx")
- Prefix / suffix
- Dictionary lookups → gazetteers
- Neighboring words
- Tag bigrams (previous tag + current word)

CRFs = logistic regression for each token, but with **global sequence consistency**.

Although the idea of what features to use is done by the system designer by hand, the specific features are automatically populated by using feature templates as we feature templates. Here are some templates that only use information from (y_{i-1}, y_i, X, i) :

- $\langle y_i, x_i \rangle$
- $\langle y_i, y_{i-1} \rangle$
- $\langle y_i, x_{i-1}, x_{i+2} \rangle$

These templates automatically populate the set of features from every instance in the training and test set. Thus for our example "Janet/NNP will/MD back/VB the/DT bill/NN", when x_i is the word back, the following features would be generated and have the value 1 (we've assigned them arbitrary feature numbers):

- $f_{3743} : y_i = VB \text{ and } x_i = \textit{back}$
- $f_{156} : y_i = VB \text{ and } y_{i-1} = MD$
- $f_{99732} : y_i = VB \text{ and } x_{i-1} = \textit{will} \text{ and } x_{i+2} = \textit{bill}$



INFERENCE FOR CRFS

Plug into Viterbi!

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) \sum_{k=1}^K w_k f_k(y_{t-1}, y_t, X, t)$$

$$1 \leq j \leq N, 1 < t \leq T$$

Evaluate using standard metrics.

Information Extraction: Applications

After tagging, we can extract richer meaning.

Relation Extraction

Example:

- “[ORG United] is a unit of [ORG UAL Corp]”
- “[PER Tim Wagner] works for them”
- “[LOC Chicago] to [LOC Dallas]” (route relation)

Relations = triples: $(entity_1, relation, entity_2)$

These can form knowledge graphs.

Approaches:

- **Pattern-based** (Hearst patterns)
Example: “X is located in Y”
- **Supervised classifiers** for relation types
- **Distant supervision** using known KB triples

Challenges:

- Domain shift → models break
- Requires labeled data
- Patterns are brittle

RDF Triples

Resource Description Framework:

- (Evoluo, located-in, Eindhoven)

Can be extracted via:

- patterns
- supervised learning
- distant supervision

This forms the basis of **knowledge graphs** (e.g., Freebase, Wikidata).

Event Extraction

Hard task: find events + their participants + temporal ordering.

Example:

- [EVENT said]
- [EVENT increase]
- [EVENT competes]

Also involves time detection:

- absolute times (“April 24 1916”)
- relative times (“next semester”)
- durations (“for two years”)

Challenges:

- Natural language expresses time very inconsistently
- Requires normalizing into structured formats
- Current systems struggle heavily

Why This Matters

Before moving to deep learning next week, even if neural models now outperform these methods, we must understand **the classical foundations**, because neural sequence models often *implicitly* do what HMMs/CRFs do *explicitly*.

Sequence modeling teaches:

- what structured prediction is

- how language carries dependencies
- how predictions can depend on other predictions

This conceptual grounding prepares you for RNNs, LSTMs, and Transformers.

In the previous lectures,

- We looked at how language might be noisy as input.
- We discussed several way to represent and model language: count and prediction-based, and sequential.
- We looked at framing prediction tasks around those respective representations.

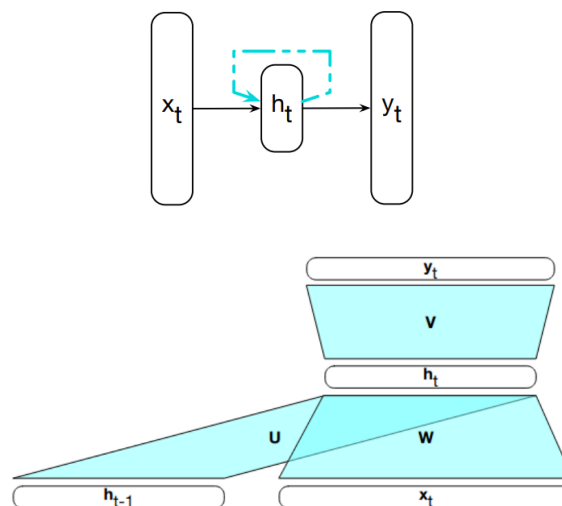
Today, we discuss Deep Learning: end-to-end **representation learning** and **predictions** in one **black box**.

Deep Learning

Recurrent Neural Networks

A recurrent neural network (RNN) is any network that contains a cycle within its network connections, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input.

RNNs were the first major neural architecture for handling **sequential data** (like text). Unlike Feed-Forward Networks (FFNs), which assume independent inputs, RNNs have a "memory" of previous inputs.



- **Mechanism:** The network passes a **hidden state (h)** from the previous time step to the current time step.
- **Lookback:** It encodes earlier steps in the sequence to influence future decisions.
- **Shared Weights:** The weight matrices (U, V, W) are **shared** across all time steps (they don't change per word).

The Math

At time step t :

1. **Input:** x_t (current word embedding)
2. **Previous Hidden State:** h_{t-1}
3. Current Hidden State Calculation: $h_t = g(Uh_{t-1} + Wx_t)$
 - U : Weight matrix for the hidden state (context).
 - W : Weight matrix for the input.
 - g : Non-linearity function (ReLU or tanh)
4. Output Calculation: $\hat{y}_t = \text{softmax}(Vh_t)$
 - V : Weight matrix mapping hidden state to output (vocabulary size).

The dimensionalities are as follows, given d_{in}, d_h, d_{out} :

- $W \in \mathbb{R}^{d_h \times d_{in}}$
- $U \in \mathbb{R}^{d_h \times d_h}$
- $V \in \mathbb{R}^{d_{out} \times d_h}$

RNNs as Language Models

$$\mathbf{e} = [\mathbf{Ex}_{t-3}; \mathbf{Ex}_{t-2}; \mathbf{Ex}_{t-1}]$$

$$\mathbf{h} = \sigma(\mathbf{We} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{Uh}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

$$\mathbf{e}_t = \mathbf{Ex}_t$$

$$\mathbf{h}_t = g(\mathbf{Uh}_{t-1} + \mathbf{We}_t)$$

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{Vh}_t)$$

Window-based Neural Language Model (N-gram NN)

Input embeddings

$$\mathbf{e} = [Ex_{t-3}; Ex_{t-2}; Ex_{t-1}]$$

- $x_{t-3}, x_{t-2}, x_{t-1}$ are one-hot vectors for the previous 3 words.
- E is the embedding matrix.
- You concatenate these embeddings into one long vector $\mathbf{e}$.

Then we have the hidden layer $\mathbf{h}$ (a standard feedforward layer), and the output (the logits) on which we apply the softmax to get the predicted distribution. This gives the probability of each word in the vocabulary being the *next* word.

The probability the model assigns to the true next word is the corresponding coordinate in the predicted softmax vector.

And the probability of an entire sequence is:

$$P(w_{1:n}) = \prod_{i=1}^n y_i[w_i]$$

That's how language models define sequence likelihood.

RNN Language Model (the modern approach)

Input embedding

$$\mathbf{e}_t = Ex_t$$

Same idea, but only one word at a time.

Hidden state update (recurrent step): $\mathbf{h}_t = g(\mathbf{Uh}_{t-1} + \mathbf{We}_t)$

This is the key difference:

- The hidden state **carries memory of all past words**
- Not just the last 3

Output distribution

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{Vh}_t)$$

This gives next-word probabilities: $P(w_{t+1} \mid w_1, \dots, w_t) = \hat{y}_t$

So $\hat{\mathbf{y}}_t$ is the predicted probability distribution over the entire vocabulary $\mathbf{V}$ at time step t

$\hat{y}_t[i]$ is the model's predicted probability that the **next** word is the i -th word in the vocabulary.

For predicted distribution $\hat{\mathbf{y}}_t$ and true next word w_{t+1} [i.e.

- $y_t[w] = 1$ if $w = w_{t+1}$
- $y_t[w] = 0$ otherwise] $\rightarrow$

$$L_{CE} = - \sum_{w \in V} y_t[w] \log \hat{y}_t[w]$$

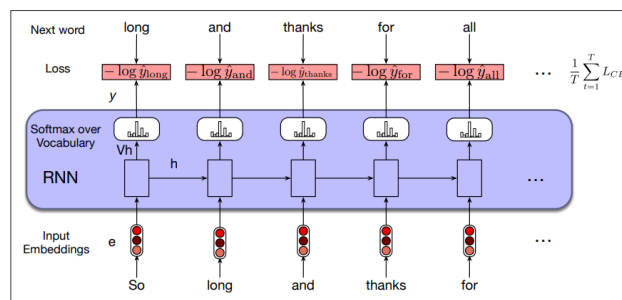
But since the target is one-hot (for example $y_t[w] = [0, 0, 0, 1, 0]$), this simplifies to:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = - \log \hat{y}_t[w_{t+1}]$$

The loss is simply the negative log probability the model assigned to the actual next word.

- **Objective:** Predict the next word w_{t+1} given sequence $w_1 \dots w_t$.
- **Loss Function:** Cross-entropy loss (L_{CE}) compares the predicted probability distribution against the actual next word (one-hot vector).
- **Teacher Forcing:** During training, the model is fed the *correct* previous word regardless of what it predicted, to prevent errors from compounding.

Putting this all together:



Each input word (one-hot) is converted to a dense vector: $e_t = Ex_t$

Example shown: the input sequence starts with “So”, then “long”, “and”, “thanks”, “for”, “all”, ...

The RNN box is unrolled through time. At each time step t the RNN receives the input embedding e_t and the previous hidden state h_{t-1} and computes: $h_t = g(Uh_{t-1} + We_t)$

This hidden state is the model’s memory / context up to time t .

From the hidden state we get logits and then a softmax: $z_t = Vh_t$, $\hat{y}_t = \text{softmax}(z_t)$

So $\hat{y}_t[i]$ is the model’s probability that the next word (at position $t+1$) is vocabulary item i .

Then for each time step you compute the negative log probability of the true next word w_{t+1} .

The overall training loss is the average across time steps (or sum):

$$L = \frac{1}{T} \sum_{t=1}^T L_t$$

So: input → embedding → RNN update → softmax → compare predicted distribution to actual next word → compute timestep loss → average losses.

The same matrix is used for input embeddings E and (usually the transpose) for the softmax’s output matrix V . Concretely, logits $z_t = Vh_t$ and embeddings $e_t = Ex_t$ use the same parameters (often $V = E^T$).

Let’s go over an example to demonstrate how a few words are ‘forwarded’ through the network to arrive at a prediction **for each timestep t** . Here, we can imagine we have the following vocabulary / index mapping V :

word	and	for	long	so	thanks
index	1	2	3	4	5

Similarly, we have a 2-dimensional (typically this is 300-1000-ish) **embedding matrix E** :

word	and	for	long	so	thanks
dim 1	0.087	0.698	0.474	0.698	0.122
dim 2	0.940	0.711	0.897	0.978	0.175

We encode the words in the input (So long) as one-hot vectors:

input	and	for	long	so	thanks
x_0	0	0	0	1	0
x_1	0	0	1	0	0

The following steps are given by:

```

function ForwardRNN( $x$ , network) :  $y$ 
     $h_0 \leftarrow 0$ 
    for  $i \leftarrow \text{length}(x)$  do
         $h_i \leftarrow g(Uh_{i-1} + Wx_i)$ 
         $y_i \leftarrow f(Vh_i)$ 
    return  $y$ 
end function

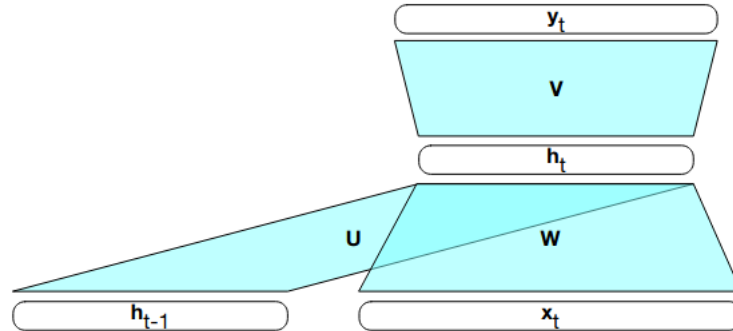
```

Above, $i = t$ (so notice that an input index matches a timestep, timestep is just RNN lingo). For each t , we select from our word embedding matrix the columns where x_n is 1. This gives the following input dimensionality $(1*d)*1$, where d is the dimensionality of the word embedding. In our case, for each t , we get $2*1$:

$$\mathbf{E}\mathbf{x}_{t_0} = \begin{bmatrix} 0.698 \\ 0.978 \end{bmatrix} \quad \mathbf{E}\mathbf{x}_{t_1} = \begin{bmatrix} 0.474 \\ 0.897 \end{bmatrix}$$

Note: $\mathbf{E}\mathbf{x}_t$ is the multiplication between $\mathbf{E}$ (embedding matrix) and x_t , transposed to be more specific, which is the one hot encoding, so $\mathbf{E}\mathbf{x}_t$ will simply be given by the embedding vector of the input x_t .

In the first step (before weights are updated via backpropagation), the weight matrices $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{V}$ are initialized randomly. To keep the example minimal, let's assume we have 2 hidden neurons (denoted as h). Remember, visually, we are doing:



Then: $\mathbf{W} \in \mathbb{R}^{d_h \times d_{in}}$, $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$, and $\mathbf{V} \in \mathbb{R}^{d_{out} \times d_h}$. Here, in is our embedding dimensionality, and out the vocabulary size. Hence, $\mathbf{W}$ is 2×2 , $\mathbf{U}$ 2×2 , and $\mathbf{V}$ 5×2 . So:

$\mathbf{W} =$

	dim 1	dim 2
h1	0.375	0.951
h2	0.732	0.599

$\mathbf{U} =$

	h1	h2
h1	0.156	0.156
h2	0.058	0.866

$\mathbf{V} =$

	h1	h2
and	0.601	0.683
for	0.021	0.970
long	0.832	0.212
so	0.182	0.183
thanks	0.304	0.525

!! We'll set the biases to 0 for this example, so we don't have to worry about them, but note that all learned parameters ($\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$) might have added biases. Remember that these parameters are initialized randomly, and do not change per timestep (they can be updated after a full pass through the data, for example).

Now we have all our parts, given the equation:

$$\begin{aligned} \mathbf{e}_t &= \mathbf{E}\mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t), \\ \hat{\mathbf{y}}_t &= \text{softmax}(\mathbf{V}\mathbf{h}_t) \end{aligned} \tag{7}$$

we can roll out our forwarding steps. At the first timestep, $\mathbf{h}_{t-1}$ is initialized to zero:

- - - - -

$$\mathbf{h}_0 = \mathbf{U} \bullet \mathbf{h}_{t-1} + \mathbf{W} \bullet \mathbf{e}_t = \begin{bmatrix} 0.156 & 0.156 \\ 0.058 & 0.866 \end{bmatrix} \bullet \begin{bmatrix} 0.000 \\ 0.000 \end{bmatrix} + \begin{bmatrix} 0.375 & 0.951 \\ 0.732 & 0.599 \end{bmatrix} \bullet \begin{bmatrix} 0.698 \\ 0.978 \end{bmatrix} = \begin{bmatrix} 1.192 \\ 1.097 \end{bmatrix}$$



Note that $\bullet$ denotes matrix multiplication ($\begin{bmatrix} a & b \end{bmatrix} \bullet \begin{bmatrix} c \\ d \end{bmatrix}$) or the dot product (multiply, sum if uneven), so one column operation looks like: $0.375 \times 0.698 + 0.951 \times 0.978 \approx 1.192$.

We'll use ReLU activations for g :

$$g(x) = \max(0, x) \quad (8)$$

If you notice, for this example, we can just take the values, as they will always be positive (😊):

$$g\left(\begin{bmatrix} 1.192 \\ 1.097 \end{bmatrix}\right) = \begin{bmatrix} 1.192 \\ 1.097 \end{bmatrix}$$

For our word prediction, we weight and map $\mathbf{h}_0$ to the output dimensionality, and pass it through our softmax:

$$f(z) = \frac{e^z}{\sum_{i=1}^K e^{z_i}}$$

This gives:

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) = f\left(\begin{bmatrix} 0.601 & 0.683 \\ 0.021 & 0.970 \\ 0.832 & 0.212 \\ 0.182 & 0.183 \\ 0.304 & 0.525 \end{bmatrix} \bullet \begin{bmatrix} 1.192 \\ 1.097 \end{bmatrix} = \begin{bmatrix} 1.466 \\ 1.089 \\ 1.224 \\ 0.418 \\ 0.938 \end{bmatrix}\right) = \begin{bmatrix} 0.293 \\ 0.201 \\ 0.230 \\ 0.103 \\ 0.173 \end{bmatrix}$$

Finally, if we compute $\arg \max_y$ we get the predicted word index: 0, which means after **So** we predict “and” (V_0), which is incorrect. Note that this is just the first pass in which we initialized all our weight matrices randomly. The network will, based on the error, tune the weights.

Anyway, that was timestep 0! Now we have an actual representation for $\mathbf{h}_0$, which we are going to pass into the next step, rather than our zero matrix before. For $\mathbf{e}_t$ we use the embedding for our second word: **long**.

$$\mathbf{h}_1 = \mathbf{U} \bullet \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t = \begin{bmatrix} 0.156 & 0.156 \\ 0.058 & 0.866 \end{bmatrix} \bullet \begin{bmatrix} 1.192 \\ 1.097 \end{bmatrix} + \begin{bmatrix} 0.375 & 0.951 \\ 0.732 & 0.599 \end{bmatrix} \bullet \begin{bmatrix} 0.474 \\ 0.897 \end{bmatrix} = \begin{bmatrix} 1.388 \\ 1.903 \end{bmatrix}$$

Note that $\mathbf{U}$ and $\mathbf{W}$ haven't updated at this step. Next, we compute the predicted word:

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) = f\left(\begin{bmatrix} 0.601 & 0.683 \\ 0.021 & 0.970 \\ 0.832 & 0.212 \\ 0.182 & 0.183 \\ 0.304 & 0.525 \end{bmatrix} \bullet \begin{bmatrix} 1.388 \\ 1.903 \end{bmatrix} = \begin{bmatrix} 2.134 \\ 1.875 \\ 1.558 \\ 0.601 \\ 1.421 \end{bmatrix}\right) = \begin{bmatrix} 0.329 \\ 0.254 \\ 0.185 \\ 0.071 \\ 0.161 \end{bmatrix}$$

Another prediction for “and” (V_0), but this time correct! To compute the loss, we sum over all individual losses. For Language Modelling, the loss is based on knowing the next word, so:

$$\frac{1}{T} \sum_{t=1}^T L_{CE} \quad \text{where : } (\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Note that $[w_{t+1}]$ refers to the index of the **ACTUAL** next word (meaning ‘long’ at step 0 and ‘and’ at step 1) indexing into the softmax matrix). This gives:

$$\frac{1}{2} \times (-\ln(0.185) + -\ln(0.329)) = 1.312$$

You can get the above with dot products, but this looks neater on paper. Note that if this is input into $\exp$, we have the perplexity of the model.

- Consider the following sentence encoding and embedding matrix:

word	a	network	neural
dim 1	0.432	0.291	0.732

input	a	network	neural
x_0	1	0	0
x_1	0	0	1
x_2	0	1	0

Calculate $\mathbf{E}\mathbf{x}_{t_0}$ and $\mathbf{E}\mathbf{x}_{t_1}$.

▼ Solution

$$\mathbf{E}\mathbf{x}_{t_0} = \begin{bmatrix} 0.432 \end{bmatrix} \quad \mathbf{E}\mathbf{x}_{t_1} = \begin{bmatrix} 0.732 \end{bmatrix}$$

- Calculate $\mathbf{h}_0$ given $\mathbf{W}$ and $\mathbf{U}$ below, using a ReLu activation:

$$\mathbf{W} = \begin{bmatrix} 0.771 \\ 0.074 \end{bmatrix} \quad \mathbf{U} = \begin{bmatrix} 0.358 & 0.116 \\ 0.863 & 0.623 \end{bmatrix}$$

▼ Solution

$$\mathbf{h}_0 = \max \left(\begin{bmatrix} 0.000 \\ 0.000 \end{bmatrix}, \begin{bmatrix} 0.358 & 0.116 \\ 0.863 & 0.623 \end{bmatrix} \bullet \begin{bmatrix} 0.000 \\ 0.000 \end{bmatrix} + \begin{bmatrix} 0.771 \\ 0.074 \end{bmatrix} \bullet \begin{bmatrix} 0.432 \end{bmatrix} \right) = \begin{bmatrix} 0.333 \\ 0.032 \end{bmatrix}$$

- Calculate $\hat{\mathbf{y}}_t$ given $\mathbf{V}$ below.

$$\mathbf{V} = \begin{bmatrix} 0.331 & 0.064 \\ 0.311 & 0.325 \\ 0.730 & 0.638 \end{bmatrix}$$

▼ Solution

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) = \text{softmax} \left(\begin{bmatrix} 0.331 & 0.064 \\ 0.311 & 0.325 \\ 0.730 & 0.638 \end{bmatrix} \bullet \begin{bmatrix} 0.333 \\ 0.032 \end{bmatrix} = \begin{bmatrix} 0.112 \\ 0.114 \\ 0.264 \end{bmatrix} \right) = \begin{bmatrix} 0.316 \\ 0.316 \\ 0.368 \end{bmatrix}$$

We take the argmax of this vector and we predict index 3 (neural). Note: given a tie, you can assume $\arg \max$ returns the first element.

Autoregressive generation: after training, to *generate* text you run the RNN step-by-step, but now at each step you feed the model's previously generated token back as the next input (not the ground-truth).

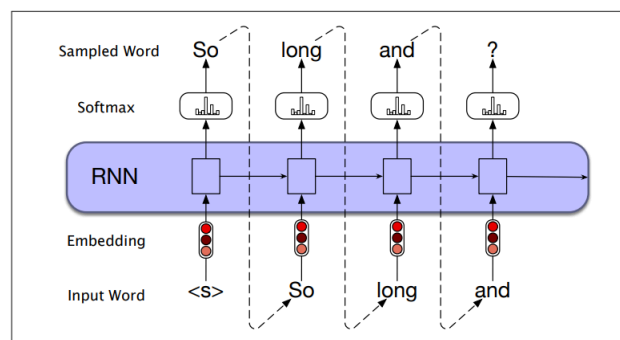


Figure 8.9 Autoregressive generation with an RNN-based neural language model.

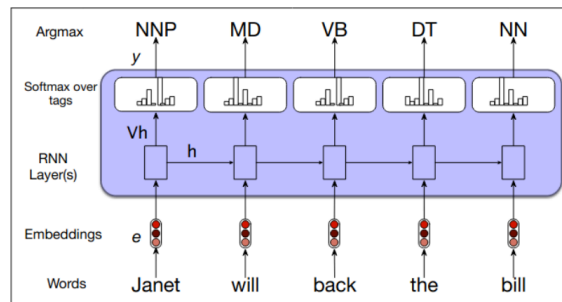
- Sample a word in the output from the softmax distribution that results from using the beginning of sentence marker, as the first input.

- Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion.
- Continue generating until the end of sentence marker, is sampled or a fixed length limit is reached.

This simple architecture underlies state-of-the-art approaches to applications such as machine translation, summarization, and question answering. The key to these approaches is to prime the generation component with an appropriate context. That is, instead of simply using to get things started we can provide a richer task-appropriate context; for translation the context is the sentence in the source language; for summarization it's the long text we want to summarize.

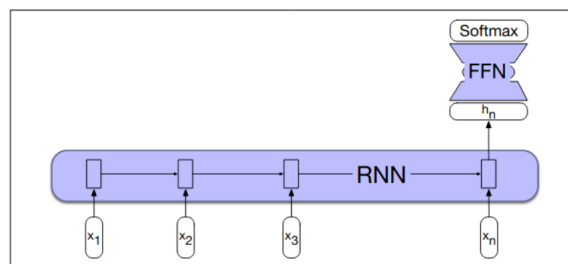
Generalizing RNN architecture to other tasks

Part-of-Speech (POS) tagging:



This works similarly to the language model objective (predicting the next word), but instead of predicting the next word in the vocabulary, the Softmax is applied over the probabilities of the tags. This allows the model to predict a tag for *every input word* in the sequence.

Classification (e.g. Sentiment Analysis or many other):



RNN can be used for sequence-to-single-label prediction, such as classifying a document or sentence.

Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

Another option, instead of using just hidden state of the last token h_n to represent pooling the whole sequence, is to use some sort of pooling function of all the hidden states h_i for each word i in the sequence. For example, $h_{mean} = \frac{1}{n} \sum_{i=1}^n h_i$, i.e. we create a representation that pools all the n hidden states by taking their element-wise mean. Or we could take the element-wise max, or even other options.

Stacked RNNs

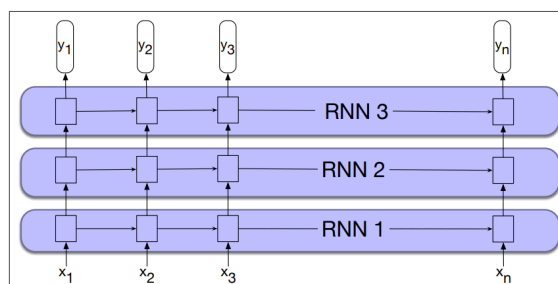


Figure 8.10 Stacked recurrent networks. The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output.

Stacked RNNs generally outperform single-layer networks. One reason for this success seems to be that the network induces representations at differing levels of abstraction across layers. It's similar to what happens in CNNs: initial layers can induce representations that serve as useful abstractions for further layers.

However, as the number of stacks is increased the training costs rise quickly.

Bidirectional RNNs

A bidirectional RNN (Schuster and Paliwal, 1997) combines two independent bidirectional RNNs, one where the input is processed from the start to the end, and the other from the end to the start.

We then concatenate the two representations computed by the networks into a single vector that captures both the left and right contexts of an input at each point in time.

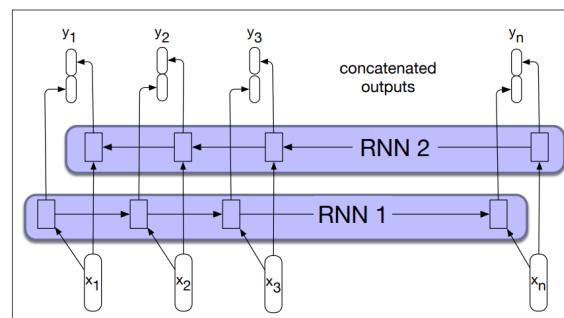


Figure 8.11 A bidirectional RNN. Separate models are trained in the forward and backward directions, with the output of each model at each time point concatenated to represent the bidirectional state at that time point.

LSTM

In practice, it is quite difficult to train RNNs for tasks that require a network to make use of information distant from the current point of processing. Despite having access to the entire preceding sequence, the information encoded in hidden states tends to be fairly local, more relevant to the most recent parts of the input sequence and recent decisions. Yet distant information is critical to many language applications.

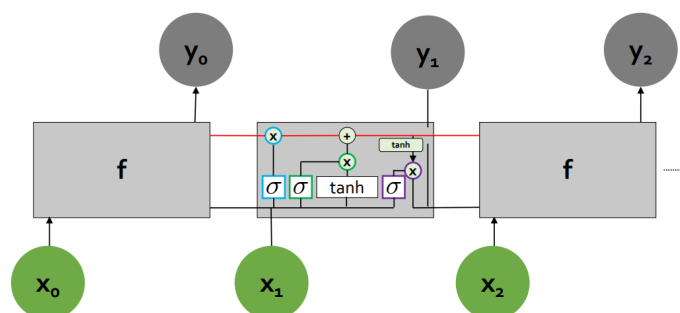
Three main problems in RNNs:

- **short context focus**
- the hidden layers are being asked to perform two tasks simultaneously: provide **information useful for the current decision**, and update and carry forward **information required for future decisions**
- **vanishing gradient problem**: the hidden layers are subject to repeated multiplications, because we need to backpropagate the error through time, and the hidden layer at time t contributes to the loss at the next time step too.

→ the most commonly used extension to RNNs is the long-short-term memory (**LSTM**) network.

LSTMs divide the context management problem into two subproblems: removing information no longer needed from the context, and adding information likely to be needed for later decision making. LSTMs accomplish this by first adding an **explicit context layer** to the architecture (in addition to the usual recurrent hidden layer), and through the use of specialized neural units that make use of gates to control the flow of information into and out of the units that comprise the network layers.

1. Cell state
2. Gates
 - Forget gate
 - Input gate
 - Output gate



The **red line** running horizontally through the diagram represents the **cell state**, which acts as the *memory conveyor belt* of the LSTM. It carries information across the entire sequence with only minor linear interactions — this allows gradients to flow more easily during training, preserving long-term information. Think of the cell state as the “**long-term memory**” of the LSTM.

Now, remember:

- the sigmoid goes from 0 to 1
- the tanh goes from -1 to 1

Forget Gate — Blue σ (“How much of the past should I remember?”)

- The **blue sigmoid** (σ) looks at the previous output y_{t-1} and the current input x_t .
- **Output of forget gate:** a vector of values between 0 and 1 → one number per element of the cell state, saying how much to keep. As you can see, there is then a $\otimes$ representing the element wise multiplication between this vector of numbers between 0 and 1 and the cell state. So, it *forgets* irrelevant parts of the previous memory.

Input Gate — Green $\sigma + \tanh$ (“What and how much new information should I add to the memory?”)

Both the green sigmoid and the tanh take in input, once again, the previous output and the current input. Based on them:

- The **green sigmoid** (σ) — *input gate* — decides **which values to update**.
- The **tanh** layer creates **candidate values** that could be added to the state.
- There is then a multiplication between the outputs of the sigmoid and the tanh, which output what and how much of the new input will be added ($\oplus$) to the long term memory.

Output Gate — Purple σ (“How much of the long term memory should I show as output?”)

- The **purple sigmoid** (σ) takes the current input and the previous output, and produces a set of control values between 0 and 1 (just like it happened in the forget gate).
- These values decide **how much of the current cell state**, after being passed through a **tanh** activation (to scale it between -1 and 1), which represents the ‘what’ (like in the input gate), should be exposed as the output y_t .
- Thus, the output gate controls how much of the cell’s internal memory becomes visible to the next layer or time step.

Remaining issue: While LSTMs solve the memory issue, they are still **sequential**. They cannot process a whole sentence in parallel (as e.g. CNNs do), preventing massive scaling.

Also, recurrence leads to information loss and training issues.

Summary:

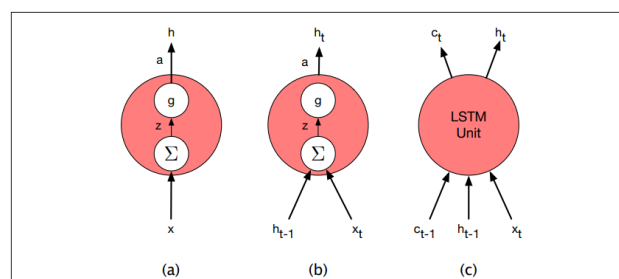


Figure 8.14 Basic neural units used in feedforward, simple recurrent networks (SRN), and long short-term memory (LSTM).

Encoder-Decoder Models

Evolution of Architectures: from RNNs to Encoder-Decoder

Before the Transformer era, the primary structure for sequential tasks like Machine Translation was the **Encoder-Decoder** model, often built using RNNs.

Encoder-decoder networks, sometimes called sequence-to-sequence networks, are models capable of generating contextually appropriate, arbitrary length, output sequences given an input sequence.

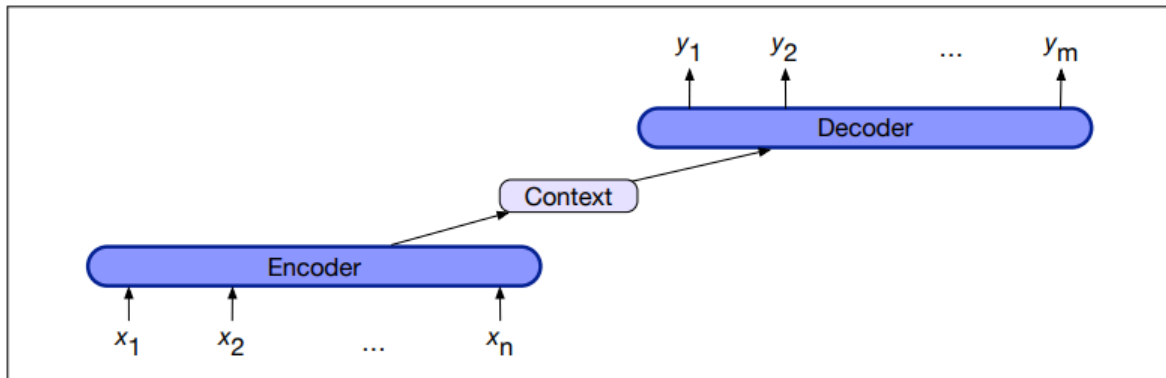
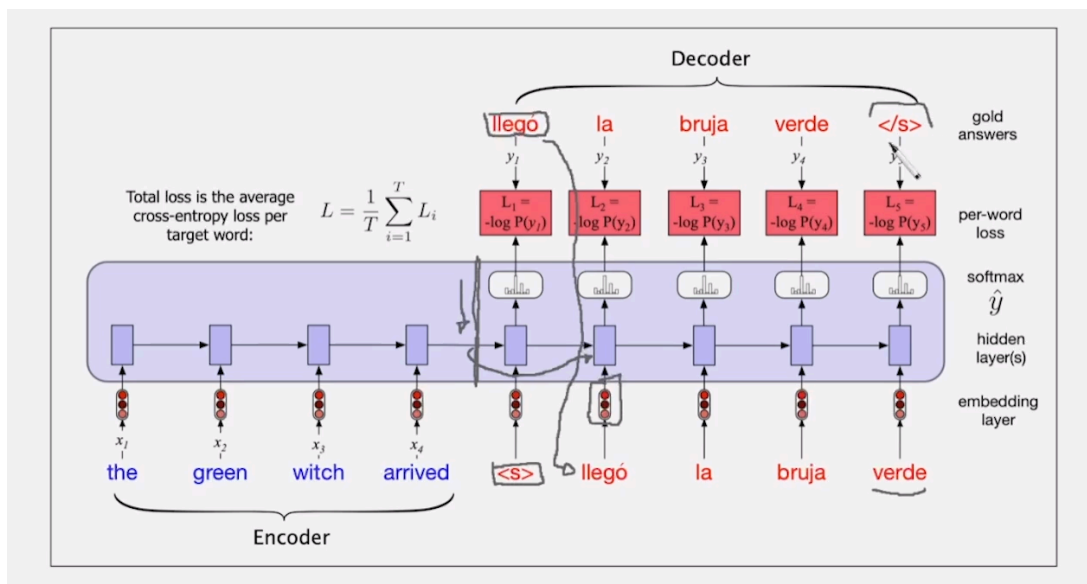


Figure 8.16 The encoder-decoder architecture. The context is a function of the hidden representations of the input, and may be used by the decoder in a variety of ways.

- **The Encoder:** Compresses the entire input sequence into a single, fixed-size **Context Vector** (usually the last hidden state).
- **The Decoder:** Takes the context vector and generates the output sequence, predicting words one at a time (it is **auto-regressive**).

Machine translation



This is two RNNs. The first one needs to encode the **source text** into the context vector. We don't care about the outputs it produces, we only care about the last hidden state h_n , basically the one corresponding to the word 'arriving', which is the last word of the source text.

After this, begins the decoding RNN. The first input is always the start of sentence $<s>$. Auto-regressively, we generate one word at a time; each unit receives in **input both the previous generated output and the context vector** h_n .

The tricky part in machine translation is that the order of the words in two languages can vary ('can you swim' → 'tu sai nuotare?'), you can see that *can* is not in the same position of *sai*, and also sometimes we don't have an exact mapping between two words, if you think about it.

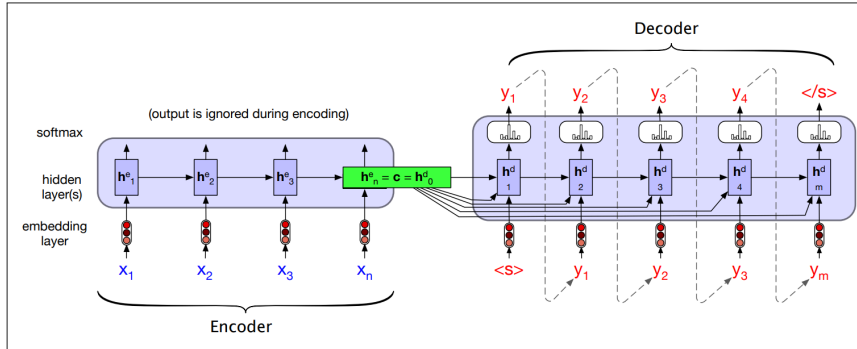


Figure 8.18 A more formal version of translating a sentence at inference time in the basic RNN-based encoder-decoder architecture. The final hidden state of the encoder RNN, h_n^e , serves as the context for the decoder in its role as h_0^d in the decoder RNN, and is also made available to each decoder hidden state.

As we said, we make the context vector c available to more than just the first decoder hidden state, to ensure that the influence of the context vector, c , doesn't wane as the output sequence is generated. We do this by adding c as a parameter to the computation of the current hidden state, using the following equation:

$$h_t^d = g(\hat{y}_{t-1}, h_{t-1}^d, c)$$

Now we can see formally the equations of this encoder-decoder model with RNNs:

- $c = h_n^e$
- $h_0^d = c$
- $h_t^d = g(\hat{y}_{t-1}, h_{t-1}^d, c)$
- $z_t = f(h_t^d)$
- $\hat{y}_t = \text{softmax}(h_t^d) \rightarrow$ vector of probabilities over the vocabulary, representing the probability of each word occurring at time t . To generate text, we sample from this distribution. For example, the greedy choice is simply to choose the most probable word to generate at each timestep.

Difference between training and inference

Note the difference between training and inference with respect to the outputs at each time step. The decoder during inference uses its own estimated output $\hat{y}_t$ as the input for the next time step x_{t+1} . Thus the decoder will tend to deviate more and more from the gold target sentence as it keeps generating more tokens.

In training, therefore, it is more common to use **teacher forcing** in the decoder, which means that we force the system to use the gold target token from training as the next input x_{t+1} , rather than allowing it to rely on the (possibly erroneous) decoder output $\hat{y}_t$. This speeds up training.

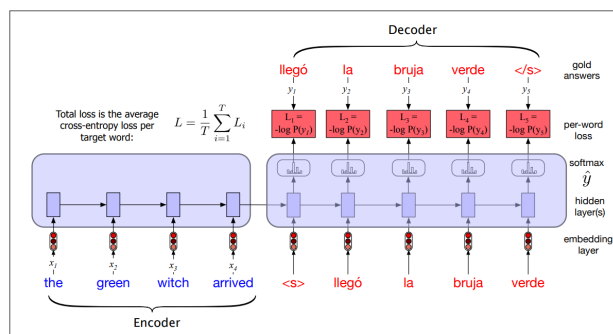


Figure 8.19 Training the basic RNN encoder-decoder approach to machine translation. Note that in the decoder we usually don't propagate the model's softmax outputs $\hat{y}_t$, but use **teacher forcing** to force each input to the correct gold value for training. We compute the softmax output distribution over $\hat{y}$ in the decoder in order to compute the loss at each token, which can then be averaged to compute a loss for the sentence. This loss is then propagated through the decoder parameters and the encoder parameters.

The Problem (The Bottleneck)

The simplicity of the encoder-decoder model is its **clean separation** of the encoder, which builds a representation of the source text, from the decoder, which uses this context to generate a target text.

In the model as we've described it so far, this context vector is h_n , the hidden state of the last (n-th) time step of the source text. This final hidden state is thus acting as a **bottleneck**: it **must represent absolutely everything about the meaning of the source text**, since the only thing the decoder knows about the source text is what's in this context vector. Cramming a long input into a single vector inevitably led to information loss, and the effect of this context vector would fade as the decoder produced more output tokens.

The Solution: Attention

The attention mechanism is a solution to the bottleneck problem, a way of attention mechanism allowing the decoder to get information from all the hidden states of the encoder, not just the last hidden state.

The idea of attention is instead to create the **single fixed-length vector c** by taking a **weighted sum of all the encoder hidden states**. In this sum, we give more importance to the parts of the source that are relevant to the token we are currently generating, i.e. the weights focus on ('**attend to**') a particular part of the source text. Attention, therefore, replaces the static context vector with one that is dynamically derived from the encoder hidden states, **different for each token in decoding**.

This context vector, c_i , is generated anew with each decoding step i and takes all of the encoder hidden states into account in its derivation.

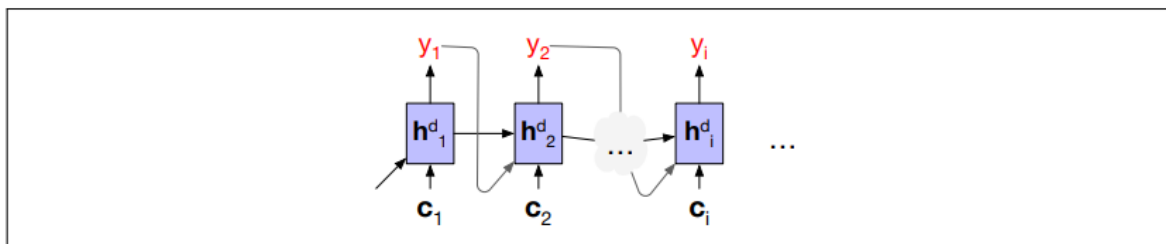


Figure 8.21 The attention mechanism allows each hidden state of the decoder to see a different, dynamic, context, which is a function of all the encoder hidden states.

$$h_i^d = g(\hat{y}_{i-1}, h_{i-1}^d, c_i)$$

How to compute how much each decoder state has to attend to the encoder states, i.e. the source words? We compute a score through dot product, which basically implements relevance as similarity. At each state i during decoding, for each encoder state j , we compute:

$$\begin{aligned} \text{score}(h_{i-1}^d, h_j^e) &= h_{i-1}^d \cdot h_j^e \\ \alpha_{ij} &= \text{softmax}(\text{score}(h_{i-1}^d, h_j^e) \forall j \in e) \\ c_i &= \sum_j \alpha_{ij} h_j^e \end{aligned}$$

By querying the encoder with the *previous* decoder hidden state h_{i-1}^d , the model uses the most current, fully-formed summary of the target sentence generated so far to decide where to focus in the source sentence.

We can see this type of attention (cross-attention) in encoder-decoder transformers (chapter 13.2 in the book).

Pre-Transformer Contextual Embeddings

Intermediate steps helped NLP progress toward deep learning:

ELMo: Used deep, bi-directional LSTMs to create **contextual embeddings**, where a word's vector representation changes based on its surrounding words.

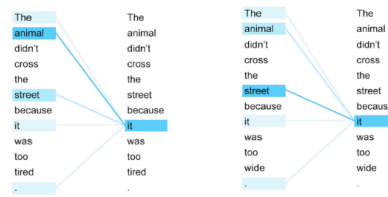
ULMFiT: Popularized **Transfer Learning**, proving that a model pre-trained on a general language task (Language Modeling) could be easily fine-tuned for improved performance on a specific downstream task.

Transformers

In this chapter we introduce the transformer, the standard architecture for building large language models. Transformer-based large language models have completely changed the field of speech and language processing. Indeed, every subsequent chapter in this textbook will make use of them. We'll focus for now on left-to-right (sometimes called causal or autoregressive) language modeling (decoder-only, basically), in which we are given a sequence of input tokens and predict output tokens one by one by conditioning on the prior context.

Attention in language:

"This GitHub page contains all the general information about the course and the study materials."

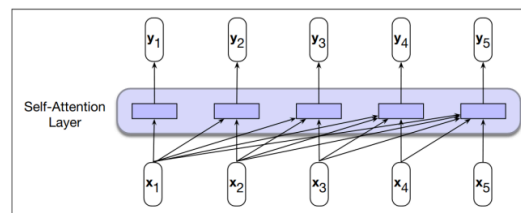


In short, what self-attention does is it moves the words in a sentence (or piece of text) closer in the word embedding. In that way, the word “bank” in the sentence “Money in the bank” will be moved closer to the word “money”. Equivalently, in the sentence “The bank of the river”, the word “bank” will be moved closer to the word “river”. That way, the modified word “bank” in each of the two sentences will carry some of the information of the neighboring words, adding context to it.

The attention mechanism **allows the model to weigh the importance of different words in an input sequence, letting it focus on relevant information for each step of its task**. It enables the model to capture relationships between words regardless of their distance, providing a context-aware understanding of language that is crucial for tasks like translation and text generation.

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \forall j \leq i \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^i \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \forall j \leq i \quad \mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$



First we get the scores as a dot product (similarity between embedding vectors), then we normalize them with a softmax to create a vector of weights α_{ij} , that tells us the proportional relevance of each vector j to the vector i .

The "Database" Analogy (Query, Key, Value)

For every word x_i , the model learns three vectors:

1. **Query (q):** What this word is looking for. $q_i = x_i W^Q$
2. **Key (k):** What this word identifies as. $k_i = x_i W^K$
3. **Value (v):** The actual content/information of the word. $v_i = x_i W^V$

Project $\mathbf{x}_i$:

$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i; \mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i; \mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i, \text{ where}$$

dims: $1 \times d$. Later: $\mathbf{W}^Q \in \mathbb{R}^{d \times d}$, $\mathbf{W}^K \in \mathbb{R}^{d \times d}$,
and $\mathbf{W}^V \in \mathbb{R}^{d \times d}$.

Actually: $\mathbf{X} \in \mathbb{R}^{N \times d}$ (embeddings),
so: $\mathbf{Q} \in \mathbb{R}^{N \times d}$, $\mathbf{K} \in \mathbb{R}^{N \times d}$,
and $\mathbf{V} \in \mathbb{R}^{N \times d}$, giving:

$$\mathbf{Q} = \mathbf{XW}^Q; \mathbf{K} = \mathbf{XW}^K; \mathbf{V} = \mathbf{XW}^V$$

which gives:

$$\text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

The Attention Mechanism

1. **Score:** Calculate relevance between word i and word j by taking the dot product of q_i and k_j .

$\text{Score} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$ (Scaled by $\sqrt{d_k}$, where d_k is the dimensionality of the key and query vectors, to prevent gradients from exploding).

For example, in the figure above, to compute score_3 we will compute three dot products: $q_3 \cdot k_1, q_3 \cdot k_2, q_3 \cdot k_3$. In fact, in causal language models (like GPT), the context is given by any of the prior words (no tokens after the token itself).

	q1•k1	—∞	—∞	—∞	—∞
	q2•k1	q2•k2	—∞	—∞	—∞
N	q3•k1	q3•k2	q3•k3	—∞	—∞
	q4•k1	q4•k2	q4•k3	q4•k4	—∞
	q5•k1	q5•k2	q5•k3	q5•k4	q5•k5
	N				

Setting weights to infinite to hide future (mask).

Thanks to this causal masking, the model can be trained in parallel, unlike in RNNs.

2. **Probabilities:** Apply Softmax to scores to get attention weights (α).
3. **Weighted Sum:** Multiply values (v) by attention weights to get the final representation.

$$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

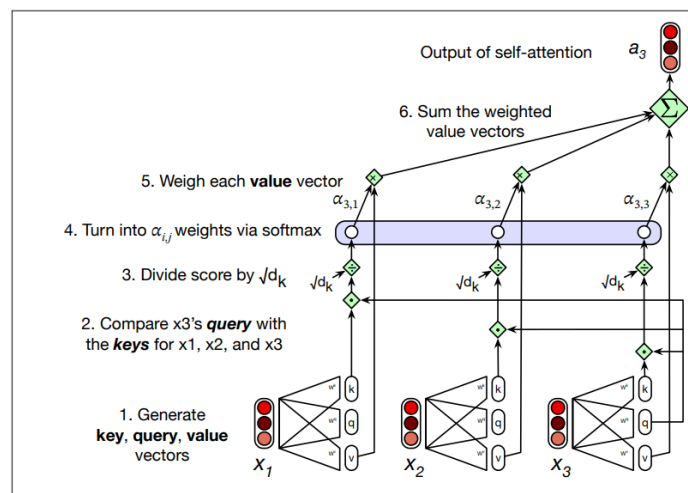


Figure 9.4 Calculating the value of a_3 , the third element of a sequence using causal (left-to-right) self-attention.

Multi-Head Attention

Instead of one set of Q, K, V , we use multiple sets ("heads").

Different heads can learn different relationships (e.g., Head 1 focuses on syntax/grammar, Head 2 focuses on semantic relationships/synonyms).

The outputs of all heads are concatenated and projected back to the model dimension.

The Transformer Block

Before seeing the full block, we have to see what layer normalization is:

$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i,$$

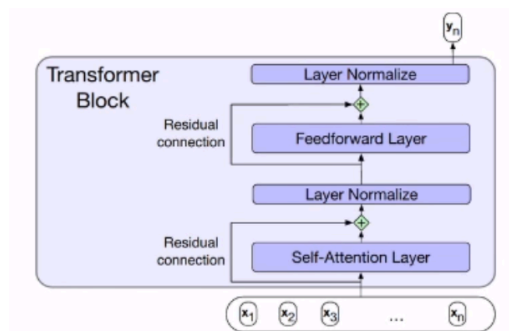
$$\sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2}$$

$$\hat{x} = \frac{(x - \mu)}{\sigma}$$

$$LayerNorm = \gamma \hat{x} + \beta$$

The normalized $\hat{x}$ is scaled by a tunable parameter γ and shifted by a tunable parameter β . These parameters are learned during training to preserve the representation power of the layer.

Now, a single block consists of:



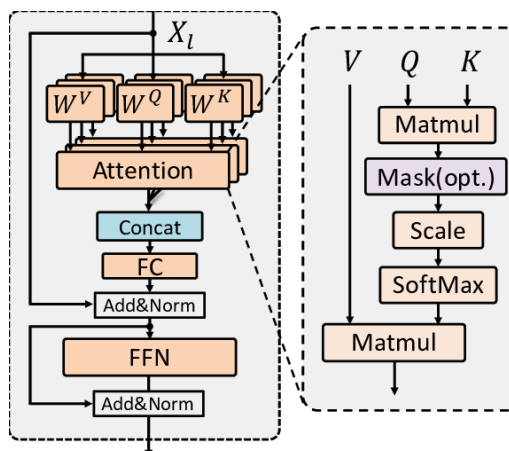
1. Multi-Head Self-Attention

2. **Add & Norm:** Residual connection (add raw embedding, i.e. pre-attention embedding, to its 'revised' version, the one post-attention) + Layer Normalization.

3. Feed-Forward Network (FFN)

It allows the model to perform local, non-linear transformations on the representation created by the Attention layer. It projects the vector to a higher dimension and then back down, enabling the model to extract and process specific features based on the context information it gathered.

4. **Add & Norm:** Another residual connection + normalization.



This picture shows multi-head attention, where there are h heads, each with its own key, query and value weight matrices. The outputs from each of the heads are concatenated and then projected down to dimension d , i.e. the same size as the input. This is done through a fully

connected layer with a matrix $W^O \in \mathbb{R}^{hd_v \times d}$, where hd_v is of course because the dimension it gets in input is h times the dimensionality of the output of a single head:

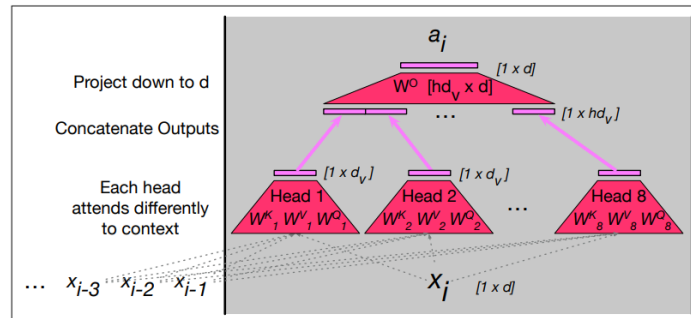


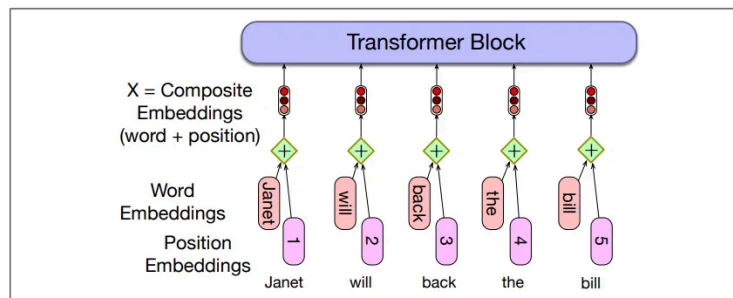
Figure 9.5 The multi-head attention computation for input x_i , producing output a_i . A multi-head attention layer has h heads, each with its own key, query and value weight matrices. The outputs from each of the heads are concatenated and then projected down to d , thus producing an output of the same size as the input.

Positional Embeddings

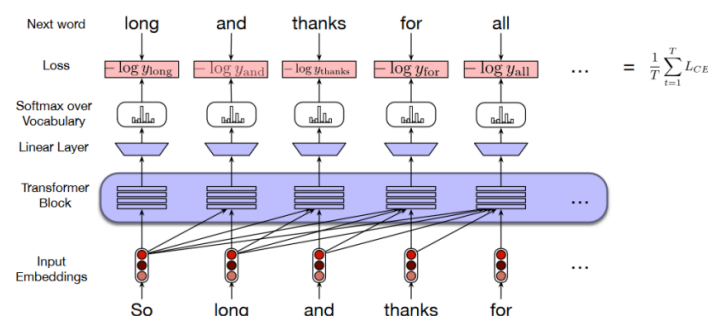
Since Transformers process all words simultaneously (parallel), they have no inherent sense of order (unlike RNNs).

Solution: Add "Positional Embeddings" to the word embeddings.

These are vectors (often using sine/cosine functions) that uniquely identify the position of a word in the sequence (1^{st} , 2^{nd} , 3^{rd} ...).

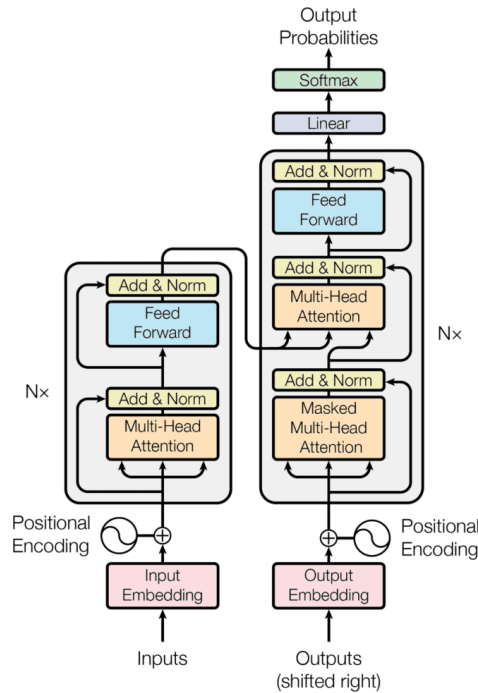


Language Modeling (Next-Word Prediction):



Encoder-Decoder in Transformers

In the previous section we often assumed a transformer decoder-only or encoder-only (even though in this second case we wouldn't have causal masking). In general, however, transformers can have this architecture and allow many combinations:



Transformers allow:

- **Decoder-only models** → autoregressive language modelling (GPT family)
- **Encoder-only models** → contextual understanding (BERT family)
- **Encoder-decoder models** → translation, summarisation (T5, original Transformer)

GPT

- A **decoder-only Transformer**
- Pre-training on language modelling
- Fine-tuning on downstream tasks (classification, QA, NLI)

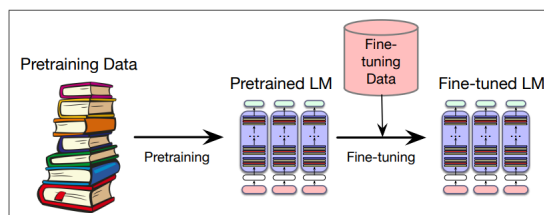


Figure 10.6 Pretraining and finetuning. A pre-trained model can be finetuned to a particular domain, dataset, or task. There are many different ways to finetune, depending on exactly which parameters are updated from the finetuning data: all the parameters, some of the parameters, or only the parameters of specific extra circuitry.

In the task of natural language inference or NLI, also called recognizing textual natural language inference entailment, a model is presented with a pair of sentences and must classify the relationship between their meanings.

A relationship of contradicts means that the premise contradicts the hypothesis; entails means that the premise entails the hypothesis; neutral means that neither is necessarily true.

- **Neutral**

- a: Jon walked back to the town to the smithy.
- b: Jon traveled back to his hometown.

- **Contradicts**

- a: Tourist Information offices can be very helpful.
- b: Tourist Information offices are never of any help.

- **Entails**

- a: I'm confused.
- b: Not all of it is very clear to me.

Supervised finetuning or SFT

SFT is often used for instruction finetuning, in which we want a pretrained language model to learn to follow text instructions, for example to answer questions or follow a command to write something. Here we create a dataset of prompts and desired responses (for example questions and their answers, or commands and their fulfillments), and we train the language model using the normal cross-entropy loss to predict each token in the instruction prompt iteratively, essentially training it to produce the desired response from the command in the prompt. It's called supervised because unlike in pretraining, where we just take any data and predict the words in it, we build the special finetuning dataset by hand, creating supervised responses to each command.

GPT-1 introduced the paradigm that still defines modern LLMs:

Generative pre-training + discriminative fine-tuning

Language Modelling and Decoding Strategies

The goal of language modelling: Predict the next word given previous context.

Evaluation: **Perplexity**.

Greedy decoding chooses the *most likely next word* each step, but this may not produce the *globally* most likely sentence.

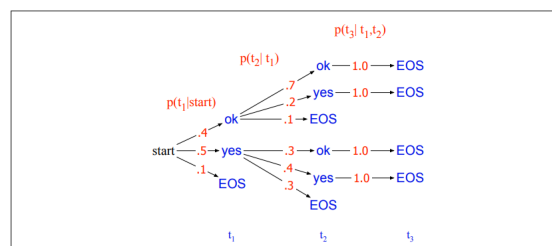


Figure 13.7 A search tree for generating the target string $T = t_1, t_2, \dots$ from vocabulary $V = \{\text{yes}, \text{ok}, \text{<eos>}\}$, showing the probability of generating each token from that state. Greedy search chooses *yes* followed by *yes*, instead of the globally most probable sequence *ok ok*.

The probability of 'ok ok' is $0.4 \times 0.7 \times 1 = 0.28$, while the probability of the chosen sentence 'yes yes' is only $0.5 \times 0.4 \times 1 = 0.2$. But because we chose a greedy search, we choose the latter.

Beam Search

Beam search is a compromise between greedy decoding (pick the single best next token every step) and exhaustive search (keep every possible sequence).

At each time step you keep the top k partial sequences (the **beam**) ranked by their cumulative score (usually log probability). Then you **expand** each beam by one token, score all expansions, and **prune** back to the top k . Repeat until you reach EOS or a max length. Beam width k controls the tradeoff: larger $k \rightarrow$ better chance of finding the highest-scoring sequence but more computation.

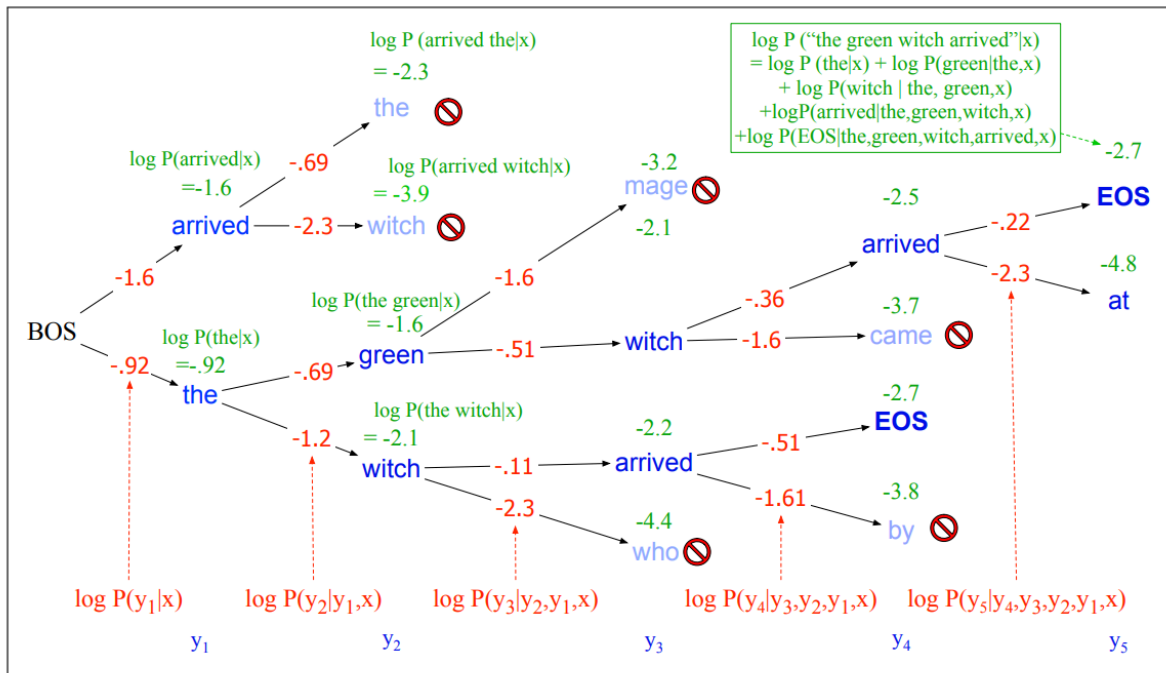


Figure 13.9 Scoring for beam search decoding with a beam width of $k = 2$. We maintain the log probability of each hypothesis in the beam by incrementally adding the logprob of generating each next token. Only the top k paths are extended to the next step.

We are adding **Log Probabilities**.

- **Red numbers:** The score of that specific single word given the context.
- **Green numbers:** The cumulative score of the entire sentence so far.
- **Rule:** $NewScore = PreviousPathScore + NewWordScore$
- **Note:** Since these are log probabilities, the numbers are negative. A score closer to 0 (e.g., -1.6) is **better** than a lower score (e.g., -3.9).

Step 1: The Start (y_1)

The model starts at **BOS** (Beginning of Sentence) and predicts the first word.

Candidates:

1. "arrived": Score -1.6
2. "the": Score -0.92

Decision: Since the beam width is 2, we keep both of these because they are the top 2 (and only) options shown.

Current Beam: ["arrived", "the"]

Step 2: Second Word (y_2)

We expand both surviving paths from Step 1 by looking at possible next words.

Expansions:

1. **From "arrived" (-1.6):**
 - ... "the": $-1.6 + (-0.69) = -2.29$ (rounded to -2.3)
 - ... "witch": $-1.6 + (-2.3) = -3.9$
2. **From "the" (-0.92):**
 - ... "green": $-0.92 + (-0.69) = -1.61$ (rounded to -1.6)
 - ... "witch": $-0.92 + (-1.2) = -2.12$ (rounded to -2.1)

Comparison & Pruning:

We compare all four new cumulative scores: $\{-2.3, -3.9, -1.6, -2.1\}$.

Winners (Top 2):

1. "the green" (-1.6)
2. "the witch" (-2.1)

Losers (Pruned):

- "arrived the" (-2.3) is lower than -2.1, so it gets the $\emptyset$.
- "arrived witch" (-3.9) is the lowest, so it gets the $\emptyset$.

Step 3: Third Word (y_3)

We now only extend the two winners from Step 2.

Expansions:

1. From "the green" (-1.6):

- ... "witch": $-1.6 + (-0.51) = -2.11$ (rounded to -2.1)
- ... "mage": $-1.6 + (-1.6) = -3.2$

2. From "the witch" (-2.1):

- ... "arrived": $-2.1 + (-0.11) = -2.21$ (rounded to -2.2)
- ... "who": $-2.1 + (-2.3) = -4.4$

Comparison & Pruning:

We compare the scores: $\{-2.1, -3.2, -2.2, -4.4\}$.

Winners (Top 2):

1. "the green witch" (-2.1)
2. "the witch arrived" (-2.2)

Losers: "mage" and "who" are pruned.

Etc.

Beam search allows us to keep a "second best" option alive just in case it turns out to be part of a better overall sentence later on.

BERT

BERT, on the other hand, uses an encoder-only (**bidirectional**) transformer architecture. It's not full language modelling, but masked language modelling.

BERT is trained via masked language modeling, masked language modeling where instead of predicting the following word, we mask a word in the middle and ask the model to guess the word given the words on both sides. This method thus allows the model to see both the right and left context.

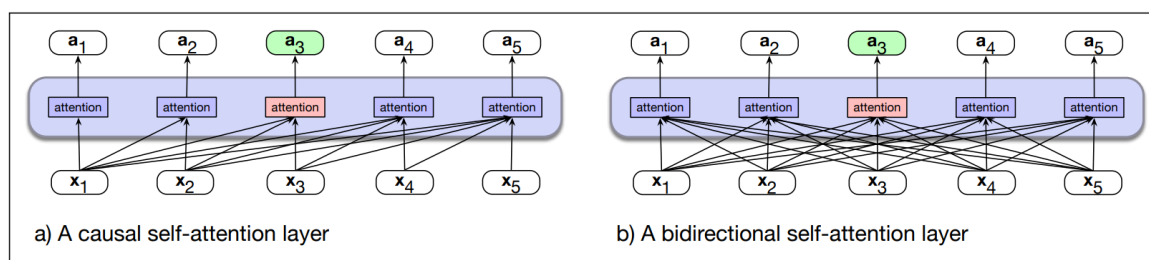


Figure 11.1 (a) The causal transformer from Chapter 9, highlighting the attention computation at token 3. The attention value at each token is computed using only information seen earlier in the context. (b) Information flow in a bidirectional attention model. In processing each token, the model attends to all inputs, both before and after the current one. So attention for token 3 can draw on information from following tokens.

The implementation is very simple! We simply remove the attention masking step that we introduced earlier.

N	$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$
	$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$
	$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$
	$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

N	$q1 \cdot k1$	$q1 \cdot k2$	$q1 \cdot k3$	$q1 \cdot k4$
	$q2 \cdot k1$	$q2 \cdot k2$	$q2 \cdot k3$	$q2 \cdot k4$
	$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$q3 \cdot k4$
	$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

Instead of predicting which words are likely to come next in this example: "The water of Walden Pond is so beautifully ____"

we're asked to predict a missing item given the rest of the sentence: "The ____ of Walden Pond is so beautifully..."

Given an input sequence with one or more elements missing, the learning task is to **predict the missing elements**. More precisely, during training the model is deprived of one or more elements of an input sequence and must generate a probability distribution over the vocabulary for each of the missing items. We then use the cross-entropy loss from each of the model's predictions to drive the learning process. This approach can be generalized to any of a variety of methods that corrupt the training input and then asks the model to recover the original input. Examples of the kinds of manipulations that have been used include masks, substitutions, reorderings, deletions, and extraneous insertions into the training text. The general name for this kind of training is called **denoising**: we corrupt (add noise to) the input in some way (by masking a word, or putting in an incorrect word) and the goal of the system is to remove the noise.

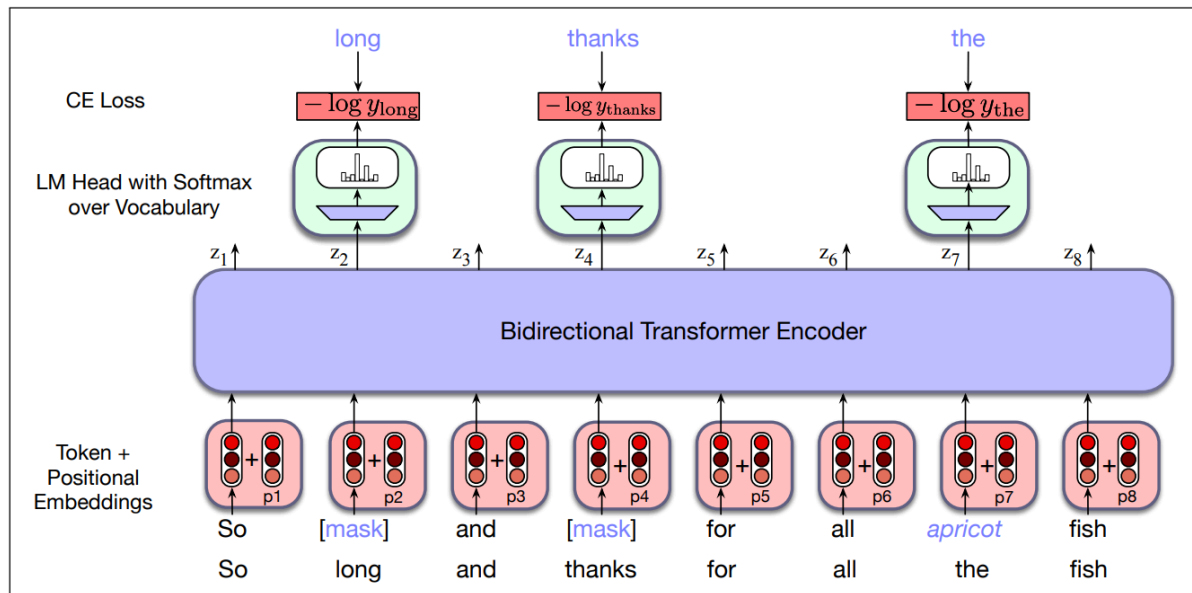


Figure 11.3 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

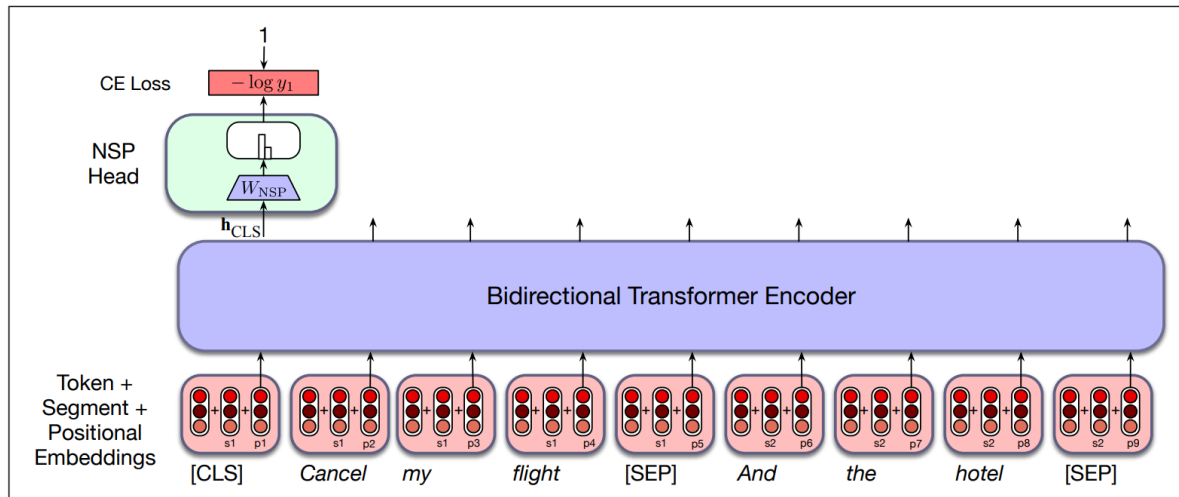
Next Sentence Prediction (NSP)

The focus of mask-based learning is on predicting missing words using the surrounding context, with the goal of learning strong word-level representations. However, many important applications require understanding the relationship **between pairs of sentences**. Examples include paraphrase detection (checking if two sentences mean the same thing), entailment (detecting whether one sentence logically follows from another or contradicts it), and discourse coherence (deciding whether two neighboring sentences fit together smoothly).

To support these kinds of tasks, some BERT-family models introduce a second training objective called **Next Sentence Prediction (NSP)**. In NSP, the model is given **pairs of sentences** and asked to predict whether the second sentence actually follows the first one in the original text, or whether it is an unrelated sentence. In BERT's training data, half of the pairs are true adjacent sentences, and the other half are random mismatches. The NSP loss measures how well the model can tell real next sentences from random ones.

To train NSP, BERT adds two special tokens to the input. After tokenizing with subword units, a **[CLS]** token is placed at the beginning of the pair, and a **[SEP]** token is placed between the two sentences and again after the second sentence. BERT also adds **segment embeddings** to mark which tokens belong to sentence A and which belong to sentence B.

During training, the output vector from the final layer corresponding to **[CLS]** is used to make the NSP prediction (0 or 1, i.e. no or yes).



Large Language Models, ChatGPT, and Prompt Engineering

From language models to large language models

We previously discussed standard language models, and now we can move into large language models.

Models like GPT-2 and GPT-3 share the **same architecture** as earlier Transformer language models. What changed is simply:

- **Much more training data** (e.g., a large portion of the Internet)
- **Much larger models** (insane numbers of parameters)
- **Massive compute for training**

That's *basically* it. Aside from small optimizations, the main improvements come from scaling: more data, more parameters, more compute. This is reminiscent of earlier "just scale it" lessons: models that didn't work well before suddenly become "magical" once scaled.

GPT (as we know it through ChatGPT) is essentially a **chat interface** on top of these models. We don't know exactly what happens "under the hood," and there are behavior adjustments we can't check.

InstructGPT

One of the biggest advances in GPT-3.5 and beyond is **how much human labor** is involved.

OpenAI realized: **if they employ humans to perform specific tasks, the model improves.**

This led to the **InstructGPT paradigm**, involving:

Step 1 – Supervised Fine-Tuning

Humans are given instructions such as: "*Explain reinforcement learning to a six-year-old.*" They produce a desired output.

This data (Prompt + Desired Output) is used to **Supervised Fine-Tune (SFT)** a pre-trained LLM (like GPT-3.5). The model learns to mimic the human demonstrator's style and content.

Step 2 – Ranking Model Outputs

This step trains a separate model that can act as a **proxy for human preference**—it learns to grade how "good" a response is.

- **Goal:** Train a **Reward Model (RM)** that scores a model output based on human-level quality and alignment.
- **Data Collection:**
 - A prompt is sampled (e.g., the same prompt as Step 1).
 - The Supervised Policy generates **several different outputs** (A, B, C, D).
 - A human labeler is then asked to **rank** these outputs from best to worst (e.g., $D > C > A > B$).
- **Model Training:** The collected ranking data (not the raw text, but the comparison: "D is better than B") is used to train the **Reward Model (RM)**. The RM is a neural network that takes a prompt and a response as input and outputs a single scalar score (the **reward**).
- **Output:** The **Reward Model (RM)**, which now possesses the ability to assign high scores to outputs humans prefer and low scores to outputs humans dislike.

Step 3 – Reinforcement Learning

This is the final, iterative stage where the language model learns to maximize the score provided by the Reward Model.

Goal: Fine-tune the language model (the Policy) to generate outputs that achieve the highest possible score from the RM.

The PPO model (the final language model) is initialized from the Supervised Policy from Step 1.

A new prompt is sampled (e.g., "Write a story about otters."). The **Policy** generates an output (e.g., "Once upon a time...").

The generated output is fed into the **Reward Model (RM)**. The RM calculates a single score, r_k (the reward).

The **Proximal Policy Optimization (PPO)** algorithm uses the reward r_k to update the Policy's weights.

- If the output received a **high reward**, PPO reinforces the probabilities of generating that sequence of tokens again.
- If the output received a **low reward**, PPO adjusts the model to avoid generating similar responses in the future.

Output: The final, human-aligned language model that can generate helpful, harmless, and preferred responses.

Prompt engineering

A prompt is a text string that a user issues to a language model to get the model to do something useful. In prompting, the user's prompt string is passed to the language model, which iteratively generates tokens conditioned on the prompt. Thus the prompt creates a context that guides LLMs to generate useful outputs to achieve some user goal. The process of finding effective prompts for a task is known as prompt engineering

A key insight from GPT research is that **prompts matter**.

Originally, researchers noticed that if you add certain triggers—like “**TL;DR**” after a long Reddit post—the model produces a summary. This sparked a huge line of research:

- Provide patterns/examples
- Let the model learn tasks by conditioning on these patterns
- See whether it **generalizes** to new inputs

This led to what some researchers jokingly call “the death of NLP”—not literally, but many feel uneasy because:

- Large language models are commercial and behind paywalls
- Even fine-tuning is now monetized (e.g., GPT Store)
- We cannot train such large models ourselves (most labs don't have the resources to train LLMs)
- Much NLP work is shifting toward **prompt engineering**

But for practitioners and data scientists, prompt engineering is extremely useful. Improving prompts can significantly improve model performance.

Prompt engineering techniques

Prompt engineering isn't about “fix my essay” use-cases; the focus is on how researchers set up prompts to make models perform tasks.

Zero-Shot

Give only an instruction, no examples.

E.g., “*Classify this text as neutral, negative, or positive.*”

The model must rely entirely on its internal knowledge.

Zero-shot often works but can be unreliable.

Few-Shot

Provide a few examples first.

Example structure:

```
Text: This is awesome / Label: negative
Text: This is bad / Label: positive
Text: What a horrible show / Label: ?
```

We hope the model picks up the pattern and applies it to the new case.

Chain-of-Thought

The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.
A: Adding all the odd numbers (9, 15, 1) gives 25. The answer is False.
The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24.
A: Adding all the odd numbers (17, 19) gives 36. The answer is True.
The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.
A: Adding all the odd numbers (11, 13) gives 24. The answer is True.
The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.
A: Adding all the odd numbers (17, 9, 13) gives 39. The answer is False.
The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.
A:

Give examples with **step-by-step reasoning** to guide the model.

Useful for logic/math tasks.

But models often latch onto superficial cues and give wrong answers. So **self-consistency** involves providing many related examples to stabilize reasoning.

Knowledge Prompting

Give the model an input plus relevant factual knowledge, helping it decide correctly.

Example:

- Statement about golf
- Provide a fact (“Golf uses least strokes wins”)
- The model can correct false statements

This helps the model focus on the right information.

Researchers try to take prompting further:

- **Retrieval-augmented prompting:** retrieve external info (e.g., from the Internet or a database) and include it in the prompt.
- **LLM-to-LLM checking:** have one model check or correct another’s output.
- **Ranking multiple answers** to measure uncertainty.
- **Running code in interpreters** to validate model-generated code.
- **Auto-refinement loops** that generate → evaluate → improve.

All of these methods improve reliability.